

身正不怕影子斜

Unioeste

Amós, Carlos e Eduarda

23 de julho de 2026

Índice

1 DP	3	3 Grafo	15
1.1 Bitmask DP	3	3.1 Bridge Tree	15
1.2 Digit DP	3	3.2 Dinic	15
1.3 Edit Distance	6	3.3 Dijkstra	17
1.4 Knapsack II	6	3.4 DSU	17
1.5 Knapsack recursivo	6	3.5 HLD - aresta	18
1.6 LCS	6	3.6 Kosaraju	19
1.7 LIS	7	3.7 LCA Binary Lifting	19
2 Estrutura de Dados	7	3.8 LCA Sparse Table	20
2.1 BIT	7	3.9 Minimum Spanning Tree	21
2.2 BIT	7	3.10 Pontos de Articulação	22
2.3 Order Statistic Tree	8	3.11 Toposort	22
2.4 SegTree Bits	8	4 Matemática	22
2.5 SegTree Iterativa	9	4.1 Combinatoria	22
2.6 SegTree Lazy	10	4.2 Cramer	23
2.7 SegTree Ultimate (Lazy Propagation, generica)	11	4.3 Divisores	23
2.8 Sparse Table	14		

4.4	EV Máximo	24	6.12	Intervalo da String Palíndromico	39
4.5	Exponenciação Rápida	24	6.13	Inversões	39
4.6	Fatoração Prima	24	6.14	Kadane	40
4.7	GCD e LCM	24	6.15	MissingCoinSUM	40
4.8	Phi Function	24	6.16	Number of Pairs (GCD e LCM)	40
4.9	Segmented Sieve	25	6.17	Path Queries 2	41
4.10	Sieve	25	6.18	Query de Segtree Offline	43
4.11	Smallest Prime Factor	25	6.19	Simspons Paradox	43
5	Primitivas	26	6.20	Steiner Tree	44
5.1	Aritmética Modular	26	6.21	Subtree Queries	45
5.2	Geometria Double	27	7	Strings	46
5.3	Geometria Int	29	7.1	KMP	46
5.4	Matrix Exponentiation	32	7.2	Manacher	46
6	Problemas	32	7.3	String Hashing	47
6.1	3 equacoes 4 variaveis	32	7.4	StringStream	49
6.2	Achar prefixos que formam a string toda	33	8	Extra	50
6.3	Artwork	33	8.1	build.sh	50
6.4	Centro de Árvore	34	8.2	compressao.cpp	50
6.5	Coin Collector	35	8.3	debug.cpp	51
6.6	Creating Multiples	36	8.4	hash.sh	54
6.7	Divisor Analysis	36	8.5	lambda.cpp	54
6.8	Dynamic Inversions	37	8.6	new.sh	54
6.9	Graph Paths I	38	8.7	prefix sum 2d.cpp	55
6.10	Graph Paths I	38	8.8	random.cpp	55
6.11	Hungaro	39	8.9	template.cpp	55

1 DP

1.1 Bitmask DP

```
//hamiltonian path
9a4 const int MAXN = 20;
2bd const int MAXM = (1<<20)+7;
3e4 const int MOD = 1e9+7;
47b vector<int> graph[MAXN];
50a int dp[MAXN][MAXM];
1a8 int n;

003 int pd(int v, int mask){
718     auto& p = dp[v][mask];
        //cout << v+1 << " " << mask << endl;
6d2     if(v == n-1){
        //cout << v+1 << " " << mask << endl;
2bf         return p = (mask ==((1<<n)-1));
c4a     }

631     if(p != -1) return p;

1a4     int ans = 0;
        //cout << graph[v].size() << endl;
284     for(auto u : graph[v]){
        //cout << u+1 << " " << mask << " " << (1<<u) << endl;
f67         if(!((mask>>u)&1)){
c1c             int newmask = mask | (1 << u);
0bc             ans = (ans + pd(u, newmask))%MOD;
58a         }
d75     }
46a     return p = ans;
b5a }

//iterativo
199 dp[1][0] = 1;
51c for (int s = 2; s < 1 << city_num; s++) {
        // only consider subsets that have the first city
ba0     if ((s & (1 << 0)) == 0) continue;
        // also only consider subsets with the last city if it's the
        full subset
8fa     if ((s & (1 << (city_num - 1))) && s != ((1 << city_num) - 1))
continue;
631     for (int end = 0; end < city_num; end++) {
ec6         if ((s & (1 << end)) == 0) continue;
        // the subset that doesn't include the current end
```

```
171         int prev = s - (1 << end);
7de         for (int j : come_from[end]) {
fc6             if ((s & (1 << j))) {
597                 dp[s][end] += dp[prev][j];
9f8                 dp[s][end] %= MOD;
7c9             }
9be         }
9ff     }
290 }

//merging subsets 0(n^3)
827 for (int mask = 0; mask < (1 << n); mask++) {
e0a     for (int submask = mask; submask != 0; submask = (submask - 1)
& mask) {
982         int subset = mask ^ submask;
        // do whatever you need to do here
70a     }
e71 }

//submasks
199 dp[1][0] = 1;
51c for (int s = 2; s < 1 << city_num; s++) {
        // only consider subsets that have the first city
ba0     if ((s & (1 << 0)) == 0) continue;
        // also only consider subsets with the last city if it's the
        full subset
8fa     if ((s & (1 << (city_num - 1))) && s != ((1 << city_num) - 1))
continue;
631     for (int end = 0; end < city_num; end++) {
ec6         if ((s & (1 << end)) == 0) continue;
        // the subset that doesn't include the current end
171         int prev = s - (1 << end);
7de         for (int j : come_from[end]) {
fc6             if ((s & (1 << j))) {
597                 dp[s][end] += dp[prev][j];
9f8                 dp[s][end] %= MOD;
7c9             }
9be         }
9ff     }
290 }
```

1.2 Digit DP

```
// TEMPLATE 1: RECURSIVA Padrão [0, R] ou [1, R]
// =====
730 int dp[20][11][2][2]; // [index][state][tight][ldz]
```

```

c3f int pd(int index, int state, int tight, int ldz) {
2e8     if(index == n) {
505         return /* condicao base */;
d0c     }

890     if (dp[index][state][tight][ldz] != -1) return
        dp[index][state][tight][ldz];

c23     int ub = tight ? r[index] - '0' : 9;
1a4     int ans = 0;

70b     for (int digit = 0; digit <= ub; digit++) {
        // if(!ldz && condicao_invalida) continue;

e16         int new_tight = tight && (digit == ub);
d3b         int new_ldz = ldz && (digit == 0);

f1b         int new_state = state; // atualiza seu estado

cf0         ans += pd(index + 1, new_state, new_tight, new_ldz);
572     }
c77     return dp[index][state][tight][ldz] = ans;
18b }

// =====
// TEMPLATE 2: RECURSIVA Range [L, R] Simultâneo
// =====
377 int dp[20][11][2][2][2]; // [index][state][above][under][ldz]

d6c int pd(int index, int state, int above, int under, int ldz) {
817     if(index == n) return /* condicao base */;

2d4     if (dp[index][state][above][under][ldz] != -1) return
        dp[index][state][above][under][ldz];

c62     int ub = above ? r[index] - '0' : 9;
f28     int lb = under ? l[index] - '0' : 0;
1a4     int ans = 0;

369     for (int digit = lb; digit <= ub; digit++) {
        // if(!ldz && condicao_invalida) continue;

394         int new_above = above && (digit == ub);
4ff         int new_under = under && (digit == lb);
d3b         int new_ldz = ldz && (digit == 0);

```

```

f1b         int new_state = state; // atualiza seu estado

a39         ans += pd(index + 1, new_state, new_above, new_under,
                    new_ldz);
510     }
430     return dp[index][state][above][under][ldz] = ans;
f74 }

// =====
// TEMPLATE 3: ITERATIVA PUSH DP
// =====
518 void solve_iterative() {
730     int dp[20][11][2][2]; // [index][last][tight][ldz]
4b9     memset(dp, 0, sizeof(dp));

a87     dp[0][10][1][1] = 1;

03b     for (int index = 0; index < n; index++) {
0c8         for (int last = 0; last <= 10; last++) {
cc3             for (int tight = 0; tight < 2; tight++) {
35d                 for (int ldz = 0; ldz < 2; ldz++) {

dcf                     int ways = dp[index][last][tight][ldz];
e6d                     if(!ways) continue;

b57                     int limit = tight ? r[index] - '0' : 9;

865                     for (int digit = 0; digit <= limit; digit++) {

5c6                         if(!ldz && last == digit) continue;

14f                             int new_tight = tight && (digit == limit);
d3b                             int new_ldz = ldz && (digit == 0);

ae9                             int next = new_ldz ? 10 : digit;

b2e                             dp[index+1][next][new_tight][new_ldz] +=
ways;

660                                 }
cde                                 }
32b                             }
f27                         }
35b                     }
77e }

7d8 int pd(int index, int sum, int above, int under){

```

```

1df      if(index == n) return (sum == 0);

a73      if (!above && !under && dp[index][sum] != -1) return
dp[index][sum];

c62      int ub = above ? r[index] - '0' : 9;
f28      int lb = under ? l[index] - '0' : 0;
1a4      int ans = 0;

369      for (int digit = lb; digit <= ub; digit++){

394          int new_above = above && (digit == ub);
4ff          int new_under = under && (digit == lb);

f98          int new_sum = (sum * 10 + digit) % M;

e79          ans = (ans + pd(index + 1, new_sum, new_above, new_under))
% MOD;
2ff      }

6aa      if (!above && !under) dp[index][sum] = ans;
ba7      return ans;
bec }

//so precisa fazer memset uma vez mas dai precisa fazer padding pra
ambos os numeros terem o mesmo numero de digitos
5db int count(int x){
d06     s = to_string(x);

eda     int pad = 19 - s.size();
a0e     if (pad > 0) s = string(pad, '0') + s;

25a     n = s.size();
272     return pd(0,0,1);
2b8 }

63d void solve(){
74f     int l, r;
5a9     cin >> l >> r;
27a     memset(dp, -1, sizeof(dp));
aba     cout << count(r) - count(l-1) << endl;
21c }

/*
b73 otimizacao super importante
7ea desse jeito, o digito menos significativo sempre esta na posicao 0

```

```

bb3      entao as posicoes dos digitos de cada numero sempre estarao na
mesma posicao
2c3      por exemplo 1234 e 12
c6f      do jeito normal ficaria
81d      1234
c20      12

47a      desse jeito fica
d93      4321
3c5      21

4be      entao a potencia de 10 em cada posicao e sempre a mesma,
permitindo so rodar o memset uma vez
c4c      */
990      vector<int> v;
7c2      int pd(int index, int tight, int mask, int ldz){
789          if (index == -1) return ((__bit_width(mask) - 1) ==
__popcount(mask));

090          if (!tight && dp[index][mask][ldz] != -1) return
dp[index][mask][ldz];

cce          int ub = tight ? v[index] : 9;
1a4          int ans = 0;
a93          int sum = 0;

70b          for (int digit = 0; digit <= ub; digit++){
e16              int new_tight = tight && (digit == ub);

d3b              int new_ldz = ldz && (digit == 0);

963              int new_mask = mask;
889              if (!new_ldz) new_mask |= (1 << digit);

6d6              ans += pd(index - 1, new_tight, new_mask, new_ldz);
739          }

0ab          if (!tight) dp[index][mask][ldz] = ans;
ba7          return ans;
1c6      }

a7e      int count(ll x){
848          v.clear();
f4c          while(x) {
d6f              v.push_back(x % 10);
6bd              x /= 10;
053          }

```

```

/* [L,R]
3cd int count(int x, int y){
d65 v2.clear();
f4c while(x) {
b64 v2.push_back(x % 10);
6bd x /= 10;
d12 }
848 v.clear();

1b8 while(y) {
da8 v.push_back(y % 10);
a53 y /= 10;
38c }
de6 while(v2.size() < v.size()) v2.push_back(0LL);
c4c */

d6c n = v.size();
726 return pd(n-1,1,0,1);
6ae }

63d void solve(){
bb3 ll r;
da1 cin >> r;
226 cout << count(r) << endl;
c01 }

0dd signed main(){
5d6 winton;
27a memset(dp, -1, sizeof(dp));
8bd int t;
f1b cin >> t;
6e6 while(t--) solve();
f37 }

```

1.3 Edit Distance

```

ac1 string s, t;
fee cin >> s >> t;
3e6 int n = s.length();
32a int m = t.length();
78a for (int i = 1; i <= n; i++){
cbc for (int j = 1; j <= m; j++){
c70 if(s[i-1] == t[j-1]) dp[i][j] = 1 + dp[i-1][j-1];
6d7 else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);

```

```

9e1 }
4b0 }

```

1.4 Knapsack II

```

7d2 int n, W;
dfd int arr[MAXN], weights[MAXN];

d09 int pd(int i, int val){
966 if(val < 0) return INF;
2e2 if(val == 0) return 0;
847 if(i == 0) return INF;

e8e if(dp[i][val] != -1) return dp[i][val];

31b return dp[i][val] = min(pd(i-1, val - arr[i]) + weights[i],
pd(i-1, val));
522 }

```

1.5 Knapsack recursivo

```

dca int n, k;
ede int pd(int w, int i){
7a7 if(w > k) return -INF;
12b if(i == n) return 0;
260 if(dp[i][w] != -1) return dp[i][w];

5f3 return dp[i][w] = max(pd(w+peso[i], i+1) + val[i], pd(w, i+1));
f97 }

```

1.6 LCS

```

ac1 string s, t;
fee cin >> s >> t;
3e6 int n = s.length();
32a int m = t.length();
78a for (int i = 1; i <= n; i++){
cbc for (int j = 1; j <= m; j++){
c70 if(s[i-1] == t[j-1]) dp[i][j] = 1 + dp[i-1][j-1];
6d7 else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);

```

```

9e1     }
4b0 }

```

1.7 LIS

```

492 for (auto x : a){
62d     int pos = upper_bound(all(lis), x) - lis.begin();
621     if (pos == (int)lis.size()) lis.push_back(x);
846     else lis[pos] = x;
498 }

```

2 Estrutura de Dados

2.1 BIT

```

// BIT de xor 1-based
// faz o xor no intervalo
//
// Complexidades:
// build - O(n)
// update - O(log(n))
// query - O(log(n))

```

```

8eb struct Bit {
1a8     int n;
116     vector<int> bit;
e86     Bit(int _n=0) : n(_n), bit(n + 1) {}
cf6     void update(int i, int x) { // xor x na posicao i
f0e         for (i; i <= n; i += i & -i) bit[i] ^= x;
2a0     }
cdb     int pref(int i) { // xor [0, i]
7c9         int ret = 0;
653         for (i; i; i -= i & -i) ret ^= bit[i];
edf         return ret;
7d1     }
9e3     int query(int l, int r) { // xor [l, r]
da4         debug(pref(r), pref(l-1));
872         return pref(r) ^ pref(l - 1);
10d     }
3d4 };

```

2.2 BIT

```

116     vector<int> bit;
1a8     int n;
5c7     void init(int _n){
8fd         n = _n;
175         bit.assign(n+1, 0);
683     }

d15     int sum(int x){
37f         int s = 0;
ca3         for(; x > 0; x -= x&(-x)){
3ef             s += bit[x];
b24         }
047         return s;
e1f     }

57c     void upd(int x, int v){
529         for(; x <= n; x += x&(-x)){
175             bit[x] += v;
5cb         }
a17     }
214 };

8eb struct Bit {
1a8     int n;
116     vector<int> bit;
e86     Bit(int _n = 0) : n(_n), bit(n + 1) {}
70f     Bit(vector<int>& v) : n(v.size()), bit(n + 1) {
78a         for (int i = 1; i <= n; i++) {
671             bit[i] += v[i - 1];
edf             int j = i + (i & -i);
b8a             if (j <= n) bit[j] += bit[i];
806         }
e89     }
cf6     void update(int i, int x) { // soma x na posicao i
b64         for (i++; i <= n; i += i & -i) bit[i] += x;
850     }

cdb     int pref(int i) { // soma [0, i]
7c9         int ret = 0;
4d3         for (i++; i; i -= i & -i) ret += bit[i];
edf         return ret;
065     }
9e3     int query(int l, int r) { // soma [l, r]
89b         return pref(r) - pref(l - 1);
e97     }

```

```
0ee };
```

2.3 Order Statistic Tree

```
//find_by_order(k): similar to find, returns the iterator corresponding
//to the k-th lowest element in the set (0-indexed).
//pode dar insert, erase, size e empty
```

```
774 #include <ext/pb_ds/assoc_container.hpp>
ca4 using namespace std;
0d7 using namespace __gnu_pbds;

4fc template <class T>
3eb using Tree =
2d0     tree<T, null_type, less<T>, rb_tree_tag,
        tree_order_statistics_node_update>;
```

```
//Tree<int> t;
//Tree<pair<int, int>> t;
```

2.4 SegTree Bits

```
// v[i] = -1 => posicao invalida (nao conta)
// v[i] = 0  => desligado
// v[i] = 1  => ligado
//
// build(n2, v2)
// query(a,b)      -> 1's em [a,b]
// update(a,b)     -> flip em [a,b]
// range_set(a,b,x) -> seta intervalo [a,b] para x (0/1)
// point_set(i, x)  -> seta posicao i para -1/0/1

fd2 namespace SegTreeBits {
34a     int tree[4*MAX];          // 1's no intervalo
692     int valid[4*MAX];         // posicoes validas no intervalo
7b3     int lazy_set[4*MAX];      // -1 = nenhum, 0 ou 1 = set pendente
882     int lazy_flip[4*MAX];     // 0/1: flip pendente
052     int n, *v;

34c     int build(int node = 1, int l = 0, int r = n-1) {
a7d         lazy_set[node] = -1;
8f7         lazy_flip[node] = 0;
893         if (l == r) {
edb             if (v[l] == -1) {
```

```
600             valid[node] = 0;
c91             tree[node] = 0;
af5         } else {
a5b             valid[node] = 1;
efa             tree[node] = v[l];
11d         }
14d         return tree[node];
d51     }
ee4     int m = (l + r)/2;
d58     build(2*node, l, m);
537     build(2*node+1, m+1, r);
d5f     valid[node] = valid[2*node] + valid[2*node+1];
b00     return tree[node] = tree[2*node] + tree[2*node+1];
1dc }

0d8 void build(int n2, int* v2) {
b77     n = n2; v = v2;
6f2     build();
b56 }

95b inline void apply_set(int node, int l, int r, int x) {
c81     tree[node] = valid[node] * x;
219     lazy_set[node] = x;
8f7     lazy_flip[node] = 0;
242 }

fa1 inline void apply_flip(int node, int l, int r) {
06b     tree[node] = valid[node] - tree[node];
c30     if (lazy_set[node] != -1) {
d5b         lazy_set[node] ^= 1;
6f5     } else {
e4d         lazy_flip[node] ^= 1;
0af     }
1b7 }

dd7 void prop(int node, int l, int r) {
8ce     if (l == r) return;
ee4     int m = (l + r)/2;
c30     if (lazy_set[node] != -1) {
d22         apply_set(2*node, l, m, lazy_set[node]);
43a         apply_set(2*node+1, m+1, r, lazy_set[node]);
a7d         lazy_set[node] = -1;
15e     }
533     if (lazy_flip[node]) {
fb9         apply_flip(2*node, l, m);
d12         apply_flip(2*node+1, m+1, r);
8f7         lazy_flip[node] = 0;
```

```

9b9      }
e8b      }

a2c      int query(int a, int b, int node = 1, int l = 0, int r = n-1) {
d5e          if (b < l || r < a) return 0;
675          if (a <= l && r <= b) return tree[node];
020          prop(node, l, r);
ee4          int m = (l + r)/2;
1db          return query(a, b, 2*node, l, m) + query(a, b, 2*node+1,
m+1, r);
05e      }

2be      int update(int a, int b, int node = 1, int l = 0, int r = n-1)
{
199          if (b < l || r < a) return tree[node];
02b          if (a <= l && r <= b) {
fd1              apply_flip(node, l, r);
14d              return tree[node];
694          }
020          prop(node, l, r);
ee4          int m = (l + r)/2;
2e4          return tree[node] = update(a, b, 2*node, l, m) + update(a,
b, 2*node+1, m+1, r);
4dd      }

1fd      int range_set(int a, int b, int x, int node = 1, int l = 0,
int r = n-1) {
199          if (b < l || r < a) return tree[node];
02b          if (a <= l && r <= b) {
98a              apply_set(node, l, r, x);
14d              return tree[node];
8dc          }
020          prop(node, l, r);
ee4          int m = (l + r)/2;
376          return tree[node] = range_set(a, b, x, 2*node, l, m) +
range_set(a, b, x, 2*node+1, m+1, r);
ea9      }

22d      int point_set(int idx, int val, int node = 1, int l = 0, int r
= n-1) {
893          if (l == r) {
a7d              lazy_set[node] = -1;
8f7              lazy_flip[node] = 0;
08c              if (val == -1) {
600                  valid[node] = 0;
c91                  tree[node] = 0;
3ca              } else {

```

```

a5b          valid[node] = 1;
a24          tree[node] = val;
d3e      }
14d          return tree[node];
718      }
020      prop(node, l, r);
ee4      int m = (l + r)/2;
dbc      if (idx <= m) point_set(idx, val, 2*node, l, m);
cac      else point_set(idx, val, 2*node+1, m+1, r);
d5f      valid[node] = valid[2*node] + valid[2*node+1];
b00      return tree[node] = tree[2*node] + tree[2*node+1];
63c      }
b96  }
```

2.5 SegTree Iterativa

```

fb1 namespace SegTree {
1a8     int n;
8f8     int pot2;
562     vector<int> tree;

0d8     void build(int n2, int* v2) {
1e3         n = n2;
fa5         pot2 = 1;
2c6         while (pot2 < n) pot2 <= 1;
26a         tree.assign(2 * pot2, 0);
4e2         for (int i = 0; i < n; ++i) tree[pot2 + i] = v2[i];
813         for (int i = pot2 - 1; i >= 1; --i) tree[i] =
max(tree[2*i], tree[2*i+1]);
641     }

4ea     int query(int a, int b) {
026         if (a > b) return 0;
238         a += pot2; b += pot2;
11e         int res = 0;
cce         while (a <= b) {
16c             if (a & 1) res = max(res, tree[a++]);
410             if (!(b & 1)) res = max(res, tree[b--]);
691             a >>= 1; b >>= 1;
80e         }
b50         return res;
d03     }

cf6     void update(int i, int x) {
1a7         int p = i + pot2;
3bc         tree[p] = x;

```

```

8b8      p >>= 1;
f77      while (p >= 1) {
280          tree[p] = max(tree[2*p], tree[2*p+1]);
8b8          p >>= 1;
6f4      }
4ad  }
092 }

```

2.6 SegTree Lazy

```

// Recursiva com Lazy Propagation
// Query: soma do range [a, b]
// Update: soma x em cada elemento do range [a, b]
//
// Complexidades:
// build - O(n)
// query - O(log(n))
// update - O(log(n))
//
// build (n, vector)
// query (int a, int b)
// update (int a, int b, int x)
// se for de multiplicacao mudar o lazy pra 1 e tirar o lazy * (r-l+1)

fb1 namespace SegTree {
d18     int tree[4*MAX], lazy[4*MAX];
052     int n, *v;

34c     int build(int node=1, int l=0, int r=n-1) {
131         lazy[node] = 0;
4e3         if (l == r) return tree[node] = v[l];
ee4         int m = (l+r)/2;
19c         return tree[node] = build(2*node, l, m) + build(2*node+1,
m+1, r);
851     }
0d8     void build(int n2, int* v2) {
680         n = n2, v = v2;
6f2         build();
acb     }
dd7     void prop(int node, int l, int r) {
7eb         tree[node] += lazy[node]*(r-l+1);
17c         if (l != r) lazy[2*node] += lazy[node], lazy[2*node+1] +=
lazy[node];
131         lazy[node] = 0;
6eb     }
a2c     int query(int a, int b, int node=1, int l=0, int r=n-1) {

```

```

020         prop(node, l, r);
e09         if (a <= l and r <= b) return tree[node];
786         if (b < l or r < a) return 0;
ee4         int m = (l+r)/2;
1db         return query(a, b, 2*node, l, m) + query(a, b, 2*node+1,
m+1, r);
cf9     }
c96     int update(int a, int b, int x, int node=1, int l=0, int
r=n-1) {
020         prop(node, l, r);
9a3         if (a <= l and r <= b) {
3bd             lazy[node] += x;
020             prop(node, l, r);
14d             return tree[node];
57d         }
ad9         if (b < l or r < a) return tree[node];
ee4         int m = (l+r)/2;
dcf         return tree[node] = update(a, b, x, 2*node, l, m) +
update(a, b, x, 2*node+1, m+1, r);
eca     }
d1a }

//struct
383 struct SegTree {
d18     int tree[4*MAX], lazy[4*MAX];
1a8     int n;

037     void build(int node, int l, int r) {
131         lazy[node] = 0;
c91         tree[node] = 0;
8ce         if (l == r) return;
ee4         int m = (l+r)/2;
d58         build(2*node, l, m);
537         build(2*node+1, m+1, r);
bb1     }

5c7     void init(int _n) {
8fd         n = _n;
224         build(1, 0, n-1);
2e4     }

dd7     void prop(int node, int l, int r) {
996         if (lazy[node]) {
3d0             tree[node] = lazy[node];
579             if (l != r) {
479                 lazy[2*node] = lazy[node];

```

```

8d6         lazy[2*node+1] = lazy[node];
98e     }
131         lazy[node] = 0;
9c8     }
f1d }

7b9 void update(int a, int b, int x, int node=1, int l=0, int
r=MAX-1) {
967     if (node == 1) r = n-1;
020     prop(node, l, r);
656     if (a > r || b < l) return;
02b     if (a <= l && r <= b) {
12e         lazy[node] = x;
020         prop(node, l, r);
505         return;
34c     }
ee4     int m = (l+r)/2;
c7a     update(a, b, x, 2*node, l, m);
d1f     update(a, b, x, 2*node+1, m+1, r);
7cf     tree[node] = max(tree[2*node], tree[2*node+1]);
20d }

2e7 int query(int a, int b, int node=1, int l=0, int r=MAX-1) {
967     if (node == 1) r = n-1;
020     prop(node, l, r);
d9e     if (a > r || b < l) return 0;
675     if (a <= l && r <= b) return tree[node];
ee4     int m = (l+r)/2;
1e7     return max(query(a, b, 2*node, l, m), query(a, b,
2*node+1, m+1, r));
c35 }
5df };

```

2.7 SegTree Ultimate (Lazy Propagation, generica)

```

// Ideia: o MOTOR (struct SegTree<Node,Tag>) nunca muda. Voce so troca
// os dois
// structs pequenos Node/Tag pra escolher a operacao
// (soma/min/max/assign/etc).
// Toda a "operacao" fica no operator+ do Node e no apply()/compose do
// Tag.
//
// Contratos que Node e Tag precisam cumprir:
// Node(): elemento neutro da query (identidade do merge)
// Node(x): folha com valor x (len = 1)
// Node operator+: merge de dois nodes (tem que ser associativo)

```

```

// Node::apply(Tag,?): aplica uma atualizacao ao node (usa node.len)
// Tag(): atualizacao "vazia" (no-op)
// Tag::apply(Tag): compoe: this recebe outra tag por cima
// Tag::empty(): true se a tag e no-op (pra nao propagar a toa)
//
// Presets prontos no fim do arquivo (soma+add, max+add, min+assign,
// MOD...).
//
// Complexidades: build O(n), query/update O(log n), find_first O(log
// n)
//
// API:
// SegTree<Node,Tag> st(n); // n folhas neutras
// SegTree<Node,Tag> st(vector<T>); // build a partir de vetor
// st.update(l, r, Tag{...}); // atualiza range [l,r]
// (0-indexado)
// st.query(l, r).sum; // consulta range [l,r]
// st.pointSet(i, Node(x)); // seta a folha i
// st.find_first(l, r, pred); // busca binaria na arvore
// (ver abaixo)
//
// =====
ad1 typedef long long ll;

e7d template<typename Node, typename Tag>
383 struct SegTree {
1a8     int n;
8a5     vector<Node> tree;
067     vector<Tag> lazy;

0a9     SegTree() : n(0) {}
c7f     SegTree(int n) { assign(n); }
65f     template<typename U> SegTree(const vector<U>& v) { build(v); }

efc     void assign(int n_) { // n folhas neutras
(Node())
e95         n = n_;
f85         tree.assign(4*n, Node());
8b5         lazy.assign(4*n, Tag());
45d     }
e52     template<typename U> void build(const vector<U>& v) {
d6c         n = v.size();
f85         tree.assign(4*n, Node());
8b5         lazy.assign(4*n, Tag());
4c2         if (n) build(v, 1, 0, n-1);
abf     }

```

```

1e1 void applyTo(int node, const Tag& t) { // aplica t no node e
    acumula no lazy
23c tree[node].apply(t);
a45 lazy[node].apply(t);
aa4 }
434 void push(int node) { // empurra lazy pros
    filhos
fee if (!lazy[node].empty()) {
e7e applyTo(2*node, lazy[node]);
bc9 applyTo(2*node+1, lazy[node]);
e71 lazy[node] = Tag();
04c }
0b5 }

477 template<typename U>
a54 void build(const vector<U>& v, int node, int l, int r) {
94f if (l == r) { tree[node] = Node(v[l]); return; }
ee4 int m = (l+r)/2;
f95 build(v, 2*node, l, m);
c67 build(v, 2*node+1, m+1, r);
8fb tree[node] = tree[2*node] + tree[2*node+1];
a09 }

dc5 void update(int ql, int qr, const Tag& t) { if (n)
    update(1,0,n-1,ql,qr,t); }
112 void update(int node, int l, int r, int ql, int qr, const Tag&
    t) {
8a6 if (qr < l || r < ql) return;
665 if (ql <= l && r <= qr) { applyTo(node, t); return; }
b55 push(node);
ee4 int m = (l+r)/2;
4b1 update(2*node, l, m, ql, qr, t);
c03 update(2*node+1, m+1, r, ql, qr, t);
8fb tree[node] = tree[2*node] + tree[2*node+1];
578 }

318 Node query(int ql, int qr) { return n ? query(1,0,n-1,ql,qr) :
    Node(); }
b1e Node query(int node, int l, int r, int ql, int qr) {
9d4 if (qr < l || r < ql) return Node();
ea7 if (ql <= l && r <= qr) return tree[node];
b55 push(node);
ee4 int m = (l+r)/2;
a6e return query(2*node, l, m, ql, qr) + query(2*node+1, m+1,
    r, ql, qr);
c59 }

```

```

3e6 void pointSet(int i, const Node& val) {
    pointSet(1,0,n-1,i,val); }
460 void pointSet(int node, int l, int r, int i, const Node& val) {
c47 if (l == r) { tree[node] = val; return; }
b55 push(node);
ee4 int m = (l+r)/2;
28e if (i <= m) pointSet(2*node, l, m, i, val);
a37 else pointSet(2*node+1, m+1, r, i, val);
8fb tree[node] = tree[2*node] + tree[2*node+1];
0a2 }

// Busca binaria na arvore.
// Retorna o MENOR indice i em [ql,qr] tal que pred( merge de
    [ql..i] ) == true.
// pred tem que ser monotono nos prefixos
    (false...false,true...true). -1 se nunca.
// Ex: menor prefixo com soma >= x ->
    find_first(0,n-1,[&](const Node&nd){return nd.sum>=x;})
1e0 template<typename F> int find_first(int ql, int qr, const F&
    pred) {
db5 Node acc = Node();
82f return find_first(1, 0, n-1, ql, qr, pred, acc);
efc }
398 template<typename F>
9ec int find_first(int node, int l, int r, int ql, int qr, const
    F& pred, Node& acc) {
96e if (qr < l || r < ql) return -1;
ce3 if (ql <= l && r <= qr) {
609 Node cand = acc + tree[node];
06b if (!pred(cand)) { acc = cand; return -1; }
40c while (l < r) { // achou no bloco:
    desce pra folha
b55 push(node);
ee4 int m = (l+r)/2;
c6a Node left = acc + tree[2*node];
a68 if (pred(left)) { node = 2*node; r = m; }
119 else { acc = left; node = 2*node+1; l =
    m+1; }
7b6 }
792 return 1;
2ef }
b55 push(node);
ee4 int m = (l+r)/2;
4c1 int res = find_first(2*node, l, m, ql, qr, pred, acc);
3b7 if (res != -1) return res;
1dc return find_first(2*node+1, m+1, r, ql, qr, pred, acc);

```

```

e7c     }
3e1 };

//
=====
// mint : inteiro modular com operator overloading (o "% MOD" plugavel)
// Use como valor dentro do Node (ex: Node com mint sum). Divisao usa
// Fermat,
// entao MOD tem que ser primo.
//
=====
622 const ll MOD = 1e9 + 7;
e54 struct mint {
aa3     ll v;
639     mint(ll x = 0) { v = x % MOD; if (v < 0) v += MOD; }
dfa     mint& operator+=(const mint& o){ v += o.v; if (v >= MOD) v -=
MOD; return *this; }
d7c     mint& operator-=(const mint& o){ v -= o.v; if (v < 0) v +=
MOD; return *this; }
23e     mint& operator*=(const mint& o){ v = v * o.v % MOD; return
*this; }
bca     mint pow(ll e) const { mint r = 1, b = *this; while (e) { if
(e&1) r *= b; b *= b; e >>= 1; } return r; }
345     mint inv() const { return pow(MOD - 2); }
878     mint& operator/=(const mint& o){ return *this *= o.inv(); }
acc     friend mint operator+(mint a, const mint& b){ return a += b; }
091     friend mint operator-(mint a, const mint& b){ return a -= b; }
962     friend mint operator*(mint a, const mint& b){ return a *= b; }
5af     friend mint operator/(mint a, const mint& b){ return a /= b; }
369     friend bool operator==(const mint& a, const mint& b){ return
a.v == b.v; }
008     friend bool operator!=(const mint& a, const mint& b){ return
a.v != b.v; }
40a     friend ostream& operator<<(ostream& os, const mint& m){ return
os << m.v; }
5dd };

//
=====
// Compressor : compressao de coordenadas reutilizavel
// Compressor<int> C;
// for (x : entrada) C.add(x);
// C.build();
// int id = C.idx(x); // id comprimido 0..C.size()-1
// int tam = C.size(); // tamanho pra criar a SegTree
// int orig = C.val(id); // volta pro valor original
//

```

```

=====
67a template<typename T>
928 struct Compressor {
cff     vector<T> vals;
ab9     void add(T x) { vals.push_back(x); }
0a8     void build() {
3d9         sort(vals.begin(), vals.end());
7ee         vals.erase(unique(vals.begin(), vals.end()), vals.end());
e43     }
f84     int size() const { return (int)vals.size(); }
6a5     int idx(T x) const { return (int)(lower_bound(vals.begin(),
vals.end(), x) - vals.begin()); }
a2c     T val(int i) const { return vals[i]; }
c13 };

//
=====
// ===== PRESETS =====
// Copie UM bloco Node/Tag pra cima do "struct SegTree" e use
// SegTree<Node,Tag>.
// Node guarda 'len' (tamanho do range) => funciona com ranges nao
// uniformes,
// como os que aparecem depois de compressao de coordenada.
//
=====
/*
// ---- (1) SOMA de range + ADD em range ---- st.query(l,r).sum
7e5 struct Tag {
8bb     ll add = 0;
be7     void apply(const Tag& t){ add += t.add; }
ece     bool empty() const { return add == 0; }
ac9 };
bf2 struct Node {
ab5     ll sum = 0; int len = 0;
2dd     Node() {}
42c     Node(ll x): sum(x), len(1) {}
84a     void apply(const Tag& t){ sum += t.add * len; }
3b7     friend Node operator+(const Node& a, const Node& b){
638         Node r; r.sum = a.sum + b.sum; r.len = a.len + b.len;
return r;
494     }
b0e };

// ---- (2) MAX de range + ADD em range ---- st.query(l,r).mx
7e5 struct Tag {
8bb     ll add = 0;

```

```

be7     void apply(const Tag& t){ add += t.add; }
ece     bool empty() const { return add == 0; }
ac9 };

bf2 struct Node {
e83     ll mx = LLONG_MIN; int len = 0;
2dd     Node() {}
2b8     Node(ll x): mx(x), len(1) {}
c4f     void apply(const Tag& t){ mx += t.add; }
3b7     friend Node operator+(const Node& a, const Node& b){
ad9         Node r; r.mx = max(a.mx, b.mx); r.len = a.len + b.len;
        return r;
dbb     }
765 };

// ---- (3) MIN de range + ASSIGN em range ---- st.query(l,r).mn
7e5 struct Tag {
f2a     ll val = 0; bool set = false;
afd     void apply(const Tag& t){ if (t.set) { val = t.val; set =
        true; } }
881     bool empty() const { return !set; }
2eb };
bf2 struct Node {
e4e     ll mn = LLONG_MAX; int len = 0;
2dd     Node() {}
77d     Node(ll x): mn(x), len(1) {}
127     void apply(const Tag& t){ if (t.set) mn = t.val; }
3b7     friend Node operator+(const Node& a, const Node& b){
de4         Node r; r.mn = min(a.mn, b.mn); r.len = a.len + b.len;
        return r;
624     }
a86 };
// atualizar: st.update(l, r, Tag{x, true});

// ---- (4) SOMA + ADD com MOD (usa mint) ---- st.query(l,r).sum
7e5 struct Tag {
68f     mint add = 0;
be7     void apply(const Tag& t){ add += t.add; }
4d8     bool empty() const { return add == mint(0); }
c14 };
bf2 struct Node {
a6a     mint sum = 0; int len = 0;
2dd     Node() {}
3f2     Node(mint x): sum(x), len(1) {}
479     void apply(const Tag& t){ sum += t.add * mint(len); }
3b7     friend Node operator+(const Node& a, const Node& b){
638         Node r; r.sum = a.sum + b.sum; r.len = a.len + b.len;
        return r;

```

```

494     }
899 };

// ---- (5) SOMA + ASSIGN em range ---- st.query(l,r).sum ; update:
        Tag{x,true}
7e5 struct Tag {
f2a     ll val = 0; bool set = false;
afd     void apply(const Tag& t){ if (t.set) { val = t.val; set =
        true; } }
881     bool empty() const { return !set; }
2eb };
bf2 struct Node {
ab5     ll sum = 0; int len = 0;
2dd     Node() {}
42c     Node(ll x): sum(x), len(1) {}
0a9     void apply(const Tag& t){ if (t.set) sum = t.val * len; }
3b7     friend Node operator+(const Node& a, const Node& b){
638         Node r; r.sum = a.sum + b.sum; r.len = a.len + b.len;
        return r;
494     }
d97 };
c4c */

```

2.8 Sparse Table

```

// Resolve RMQ
//
// Complexidades:
// build - O(n log(n))
// query - O(1)

cca namespace sparse {
d28     int m[LOG+1][MAX], n;
61c     void build(int n2, int* v) {
1e3         n = n2;
78e         for (int i = 0; i < n; i++) m[0][i] = v[i];
a1c         for (int j = 1; (1<<j) <= n; j++) for (int i = 0; i+(1<<j)
            <= n; i++)
5d5             m[j][i] = min(m[j-1][i], m[j-1][i+(1<<(j-1))]);
cae     }
4ea     int query(int a, int b) {
ee5         int j = __builtin_clz(1) - __builtin_clz(b-a+1);
dc3         return min(m[j][a], m[j][b-(1<<j)+1]);
fba     }
3e1 }

```

```

67a template <typename T>
7e9 struct SparseTable {
5a6     vector<vector<T>> m;

407     SparseTable(vector<T>& v) {
3d2         int n = v.size();
a12         int max_log = 32 - __builtin_clz(n);
d47         m.assign(max_log, vector<T>(n));
78e         for (int i = 0; i < n; i++) m[0][i] = v[i];
0de         for (int j = 1; j < max_log; j++) {
ed9             for (int i = 0; i + (1<<j) <= n; i++) {
5d5                 m[j][i] = min(m[j-1][i], m[j-1][i+(1<<(j-1))]);
642             }
a87         }
319     }

0ad     T query(int a, int b) {
ee5         int j = __builtin_clz(1) - __builtin_clz(b-a+1);
dc3         return min(m[j][a], m[j][b-(1<<j)+1]);
c89     }
4a5 };

// Sparse Table para Bitmasks
// Utiliza OR bit a bit (|) para combinar conjuntos ou atributos de um
// intervalo.
// Muito util para problemas com restricoes pequenas (ex: K <= 64),
// permitindo
// substituir K Sparse Tables por apenas 1 (ex: usando unsigned long
// long).

67a template <typename T>
0ff struct SparseTableMask {
5a6     vector<vector<T>> m;

b3e     SparseTableMask(vector<T>& v) {
3d2         int n = v.size();
a12         int max_log = 32 - __builtin_clz(n);
d47         m.assign(max_log, vector<T>(n));
78e         for (int i = 0; i < n; i++) m[0][i] = v[i];
0de         for (int j = 1; j < max_log; j++) {
ed9             for (int i = 0; i + (1<<j) <= n; i++) {
f66                 m[j][i] = (m[j-1][i] | m[j-1][i+(1<<(j-1))]);
776             }
e71         }
7f1     }

0ad     T query(int a, int b) {
ee5         int j = __builtin_clz(1) - __builtin_clz(b-a+1);

```

```

8df         return (m[j][a] | m[j][b-(1<<j)+1]);
7ed     }
991 };

```

3 Grafo

3.1 Bridge Tree

```

9fa int t, c, low[MAX], pre[MAX], visited[MAX], comp[MAX];
7b4 stack<int> st;
375 vector<int> graph[MAX], compgraph[MAX];

955 void dfs(int v, int p){
cda     visited[v] = 1;
98b     low[v] = pre[v] = ++t;
30c     st.emplace(v);
284     for (auto u : graph[v]){
264         if (!visited[u]){
412             dfs(u,v);
c80             low[v] = min(low[v], low[u]);
77d         }
4e6         else {
2da             if (u == p) continue;
39b             low[v] = min(low[v], pre[u]);
acd         }
11d     }
cf1     if (low[v] == pre[v]){ //achamos uma bridge
6a4         c++;
2f4         int cur;
016         do {
596             cur = st.top();
25a             st.pop();
524             comp[cur] = c;
cca             compgraph[c].push_back(cur);
583         } while (cur != v);
b04     }
6fc }

```

3.2 Dinic

```

// 0(min(m * max_flow, n^2 m))

```

```

// Grafo com capacidades 1: 0(min(m sqrt(m), m * n^(2/3)))
// Todo vertice tem grau de entrada ou saida 1: 0(m sqrt(n))

14d struct Dinic {
670     struct edge {
358         int to, cap, rev, flow;
7f9         bool res;
bdc         edge(int to_, int cap_, int rev_, bool res_) : to(to_),
cap(cap_), rev(rev_), flow(0), res(res_) {}
f70     };
d8f     vector<vector<edge>> graph;
afd     vector<int> level, beg;
1a8     int n;
805     int fluxo;

489     Dinic(int n) : n(n), graph(n), level(n), beg(n), fluxo(0) {}

087     void add(int a, int b, int c) {
41b         graph[a].emplace_back(b, c, (int)graph[b].size(), false);
679         graph[b].emplace_back(a, 0, (int)graph[a].size() - 1,
true);
036     }

123     bool bfs(int s, int t) {
224         fill(level.begin(), level.end(), -1);
9ef         fill(beg.begin(), beg.end(), 0);
ec8         level[s] = 0;
26a         queue<int> q;
08b         q.push(s);
14d         while (!q.empty()) {
e4a             int u = q.front();
833             q.pop();
d64             for (auto &e : graph[u]){
588                 if (level[e.to] != -1 or (e.flow == e.cap))
continue;
0ab                 level[e.to] = level[u] + 1;
6f4                 q.push(e.to);
746             }

f76         }
fcb         return level[t] >= 0;
a4e     }

036     int dfs(int v, int t, int f = INF) {
4ff         if (v == t or !f) return f;

af4         for (int &i = beg[v]; i < (int)graph[v].size(); i++) {

```

```

c12         auto &e = graph[v][i];
2da         if (level[e.to] != level[v] + 1) continue;

81f         int d = dfs(e.to, t, min(f, e.cap - e.flow));

53a         if (!d) continue;
60b         e.flow += d;
238         graph[e.to][e.rev].flow -= d;
be2         return d;
a5b     }
bb3     return 0;
d3f }

ff1     int max_flow(int s, int t) {
8ce         while (bfs(s, t)) {
9ee             while (int f = dfs(s, t)) fluxo += f;
df8         }
7f3         return fluxo;
438     }

3ef     vector<pair<int, int>> get_cut(int s, int t) {
d93         if (!fluxo) max_flow(s, t);
68c         vector<pair<int, int>> cut;
afd         vector<int> vis(graph.size(), 0), st = {s};
321         vis[s] = 1;
3c6         while (st.size()) {
b17             int u = st.back(); st.pop_back();
665             for (auto e : graph[u]) if (!vis[e.to] and e.flow <
e.cap){
c17                 vis[e.to] = 1, st.push_back(e.to);
29b             }
388         }
8ec         for (int i = 0; i < graph.size(); i++) for (auto e :
graph[i]){
9d2             if (vis[i] and !vis[e.to] and !e.res)
cut.emplace_back(i, e.to);
080         }
d1b         return cut;
022     }

//funcao para achar todos os caminhos de arestas
disjuntas(cada aresta so aparece em um caminho)
1bb     vector<vector<int>> get_paths(int s, int t){
d93         if(!fluxo) max_flow(s,t);
437         vector<vector<int>> paths;
331         vector<int> ptr(n, 0);

```

```

9af         for (int k = 0; k < fluxo; k++) {
1ca             vector<int> path = {s};
299             int cur = s;

c30             while (cur != t) {
666                 bool adv = false;
a04                 for (int &i = ptr[cur]; i <
(int)graph[cur].size(); i++) {
ee0                     auto &e = graph[cur][i];
8fa                     if (e.flow > 0 && !e.res) {
518                         e.flow--;
8f3                         cur = e.to;
73e                         path.push_back(cur);
8ef                         adv = true;
c2b                         break;
ce4                     }
fb0                 }

9fb                 if (!adv) break;
d62             }
8fe             if (cur == t) paths.push_back(path);
411         }

8b4         return paths;
d6a     }

51f };

```

3.3 Dijkstra

```

861 int dist[MAX]
d9c void dijkstra(int st){
fe1     priority_queue<pair<int,int>> pq;
260     for (int i = 0; i < n; i++)dist[i]=LLONG_MAX;
d89     dist[st] = 0;
56c     pq.push({0,st});
502     while(!pq.empty()){
115         auto [d, v] = pq.top();
842         d = -d;
716         pq.pop();
dd8         if (d != dist[v]) continue;
940         for (auto [u,w] : graph[v]){
ac8             int novadistancia = d + w;
bd8             if (novadistancia < dist[u]) {
d96                 dist[u] = novadistancia;
030                 pq.push({-novadistancia, u});

```

```

c98         }
379     }
133 }
6f3 }

```

3.4 DSU

```

d56 struct DSU{
1a8     int n;
7d2     vector<int> p, r;
5c7     void init(int _n){
8fd         n = _n;
1c4         p.resize(n);
7e4         r.resize(n);
603         for(int i = 0; i < n; i++){
f13             p[i] = i;
fd1             r[i] = 0;
606         }
55e     }

57b     int find(int x){
c8c         return p[x] == x ? x : p[x] = find(p[x]);
2d3     }

765     bool unite(int a, int b){
bca         a = find(a);
b88         b = find(b);
5c2         if(a == b) return false;
dac         if(r[a] == r[b]) r[a]++;
d1b         if(r[a] > r[b]){
264             p[b] = a;
4e6             r[a]++;
db5         }else{
84b             p[a] = b;
f6c             r[b]++;
c3d         }
8a6         return true;
632     }
31c }

```

```

d56 struct DSU {
825     vector<int> id, sz;

a56     DSU(int n) : id(n), sz(n, 1) { iota(id.begin(), id.end(), 0); }

```

```

0cf      int find(int a) { return a == id[a] ? a : id[a] = find(id[a]);
    }

765      bool unite(int a, int b) {
bca          a = find(a);
b88          b = find(b);
5c2          if (a == b) return false;
956          if (sz[a] < sz[b]) swap(a, b);
afa          sz[a] += sz[b];
592          id[b] = a;
8a6          return true;
ea5      }
c54 };

```

3.5 HLD - aresta

```

// SegTree de soma
// query / update de soma das arestas
//
// Complexidades:
// build - O(n)
// query_path - O(log^2 (n))
// update_path - O(log^2 (n))
// query_subtree - O(log(n))
// update_subtree - O(log(n))

// namespace SegTree { ... }

a70 namespace HLD {
af8     vector<pair<int,int>> graph[MAX];
480     int timer, pos[MAX], sz[MAX], sobe[MAX], ancestor[MAX],
        h[MAX], v[MAX];

c2a     void build_hld(int node, int p = -1, int f = 1){
f20         v[pos[node] = timer++] = sobe[node];
57c         sz[node] = 1;
50f         for (auto &i : graph[node]){
dd2             auto [u, w] = i;
2da             if (u == p) continue;
828             sobe[u] = w;
df7             ancestor[u] = node;
112             h[u] = (i == graph[node][0] ? h[node] : u);
b79             build_hld(u, node, f);
e0e             sz[node] += sz[u];

```

```

996         if (sz[u] > sz[graph[node][0].first] ||
graph[node][0].first == p){
8c8             swap(i, graph[node][0]);
ee5         }
2b5     }
469     if (p*f == -1)build_hld(h[node] = node, -1, timer = 0);
0a5 }

1f8     void build(int root = 0){
451         timer = 0;
295         build_hld(root);
cbf         SegTree::build(timer, v);
517     }

f04     int query_path(int a, int b){
2d5         if (a == b) return 0;
aa1         if (pos[a] < pos[b]) swap(a,b);
186         if (h[a] == h[b]) return SegTree::query(pos[b] + 1,
pos[a]);
587         return SegTree::query(pos[h[a]], pos[a]) +
query_path(ancestor[h[a]], b);
cd0     }

920     void update_path(int a, int b, int x){
d54         if (a == b) return;
aa1         if (pos[a] < pos[b]) swap(a,b);

1eb         if (h[a] == h[b]) return (void)SegTree::update(pos[b]+1,
pos[a], x);
046         SegTree::update(pos[h[a]], pos[a], x);
ab0         update_path(ancestor[h[a]], b, x);
10e     }

3e4     int query_subtree(int a){
b9f         if(sz[a] == 1) return 0;
f97         return SegTree::query(pos[a]+1, pos[a] + sz[a]-1);
ce2     }

acc     void update_subtree(int a, int x){
a5a         if(sz[a] == 1) return;
84f         SegTree::update(pos[a]+1, pos[a] + sz[a]-1, x);
bf2     }

7be     int lca(int a, int b){
aa1         if (pos[a] < pos[b]) swap(a,b);
0f9         return h[a] == h[b] ? b : lca(ancestor[h[a]], b);
263     }

```

```

b5d     int dist(int a, int b) {
2d5         if (a == b) return 0;
aa1         if (pos[a] < pos[b]) swap(a,b);
30a         if (h[a] == h[b]) return pos[a] - pos[b];
13e         return pos[a] - pos[h[a]] + 1 + dist(ancestor[h[a]], b);
58d     }
16d }

```

3.6 Kosaraju

```

881 int n, com;
042 vector<int> g[MAX];
58d vector<int> gi[MAX];
981 vector<int> g_scc[MAX];
c5a int vis[MAX];
ee6 stack<int> S;
a52 int comp[MAX];

1ca void dfs(int k) {
59a     vis[k] = 1;
7b1     for (int i = 0; i < g[k].size(); i++)
8d5         if (!vis[g[k][i]]) dfs(g[k][i]);

58f     S.push(k);
0a1 }

436 void scc(int k, int c) {
59a     vis[k] = 1;
52c     comp[k] = c;
309     for (int i = 0; i < gi[k].size(); i++)
bf6         if (!vis[gi[k][i]]) scc(gi[k][i], c);
b43 }

db8 void kosaraju() {
991     for (int i = 0; i < n; i++) vis[i] = 0;

158     for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

991     for (int i = 0; i < n; i++) vis[i] = 0;
d32     while (S.size()) {
70b         int u = S.top();
7de         S.pop();
1d6         if (!vis[u]) {
fd4             com++;
7d8             scc(u, com);
35b         }

```

```

96d     }
f84 }

// funcao para construir o grafo de componentes
ad1 void build_component_graph() {
687     for (int u = 0; u < n; u++) {
4d5         for (int v : g[u]) {
1ec             if (comp[u] != comp[v]) {
6d1                 g_scc[comp[u]].push_back(comp[v]);
5d9             }
b1e         }
1fb     }

//remove duplicatas
002     for (int i = 1; i <= com; i++){
0a2         sort(g_scc[i].begin(), g_scc[i].end());
1f9         if (g_scc[i].empty()) continue;
4ad         g_scc[i].erase(unique(g_scc[i].begin(), g_scc[i].end()),
g_scc[i].end());
1d4     }
6b7 }

```

3.7 LCA Binary Lifting

```

039 int n, ancestor[MAXN][LOG+1], depth[MAXN], visited[MAXN];
309 vector<vector<int>> graph;

955 void dfs(int v, int p){
cda     visited[v] = 1;
d38     ancestor[v][0] = p;

c64     for (int i = 1; i <= LOG; i++){
d14         ancestor[v][i] = ancestor[ancestor[v][i-1]][i-1];
a6e     }

284     for (auto u : graph[v]){
264         if (!visited[u]){
cf4             depth[u] = depth[v]+1;
412             dfs(u, v);
813         }
2a4     }
f1b }

c5d int lca(int v, int u){
6ed     if (depth[v] < depth[u]) swap(v,u);

4f1     for (int i = LOG-1; i >= 0; i--){

```

```

01e         if ((depth[v] - (1<<i)) >= depth[u]){
ed6             v = ancestor[v][i];
cc6         }
830     }

d77     if (u == v) return v;

4f1     for (int i = LOG-1; i >= 0; i--){
c47         if (ancestor[v][i] != ancestor[u][i]){
ed6             v = ancestor[v][i];
fb0             u = ancestor[u][i];
329         }
921     }
6c2     return ancestor[v][0];
40e }

33e struct LCA {
//one indexed shit
1c4     int n, l, cur;
47c     vector<vector<int>> adj, up;
991     vector<int> h, entry;
8c5     LCA (int n_) {
e95         n = n_;
4e0         l = 31 - __builtin_clz(n);
839         adj.resize(n + 1);
a71         up.assign(n + 1, vector<int>(l + 1));
717         h.resize(n + 1); entry.resize(n + 1);
b59         cur = 0;
521     }
50f     void add(int a, int b) {
fab         adj[a].push_back(b);
c87         adj[b].push_back(a);
386     }
17c     void dfs(int v = 1, int p = 0, int hh = 0) {
176         entry[v] = cur++;
4de         up[v][0] = p;
910         for(int i = 1; i <= l; i++) {
73a             up[v][i] = up[up[v][i - 1]][i - 1];
d67         }
9bc         for(auto u : adj[v]) {
58d             if(u != p) {
828                 dfs(u, v, hh + 1);
d5f             }
693         }
3bc         h[v] = hh;
a4d     }
7be     int lca(int a, int b) {

```

```

a6b         if(h[a] > h[b]) {
257             swap(a, b);
79b         }
14e         int dff = h[b] - h[a], cnt = 0;
d84         while(dff) {
57f             if(dff & 1) {
5be                 b = up[b][cnt];
0ce             }
f65             cnt++;
5e9             dff >>= 1;
488         }
a89         if(a == b) {
3f5             return a;
06e         }
957         for(int i = 1; i >= 0; i--) {
c84             if(up[a][i] != up[b][i]) {
3f4                 a = up[a][i];
6c0                 b = up[b][i];
568             }
dc1         }
e6f         return up[a][0];
cb7     }
699     int raise(int node, int dist) {
//if it returns 0, then it fucked up
ac9     int cnt = 0;
efd     while(dist) {
e3d         if(dist & 1) {
466             node = up[node][cnt];
0ed         }
f65         cnt++;
705         dist >>= 1;
1c0     }
192     return node;
887 }
b5d     int dist(int a, int b) {
b80         int z = lca(a, b);
421         return h[a] + h[b] - 2 * h[z];
0a0     }
434 };

```

3.8 LCA Sparse Table

```

/*
172 Fazer um euler tour na arvore e inserir todas as vezes que
    visitamos um no
6e7 Junto disso inserir a depth desse no em outro vetor

```

9f5 no final, o intervalo entre a e b sera representado por todos os
 nos que visitamos no caminho entre a e b
 55b basta pegar o o no com altura minima nesse intervalo
 31f para saber o ponto incial desse intervalo, basta pegar o tin de a
 e b

```
6e5 build O(N LOG N)
8fc query O(1)
```

b0d posso pensar em otimizar como chamo o lca guardando a ultima vez
 que o no foi chamado
 c4c */

```
4f6 int n, timer = 0, tin[MAX], depth[MAX];
ba0 vector<int> et;
227 pair<int,int> sp[MAX*2][LOG+1];
309 vector<vector<int>> graph; //0 indexado
```

```
955 void dfs(int v, int p){
8c5     tin[v] = et.size();
490     et.push_back(v);
284     for (auto u : graph[v]){
58d         if (u!=p){
cf4             depth[u] = depth[v]+1;
412             dfs(u,v);
490             et.push_back(v);
f82         }
7ac     }
6e0 }
```

```
07b void buildtable(){
96d     for (int i = 0; i < (int)et.size(); i++){
1e7         sp[i][0] = {depth[et[i]], et[i]};
bcc     }
2ae     for (int j = 1; j <= LOG; j++){
538         for (int i = 0; i + (1<<(j-1)) < (int)et.size(); i++){
d30             sp[i][j] = min (sp[i][j-1], sp[i+(1<<(j-1))][j-1]);
985         }
fd8     }
641 }
```

```
2b4 pair<int,int> query (int a, int b){
bfb     int len = b - a + 1;
5e4     int lg = 31 - __builtin_clz(len);
a57     return min(sp[a][lg], sp[b - (1<<lg) + 1][lg]);
11c }
```

```
7be int lca(int a, int b){
fb2     if (tin[a] > tin[b]) swap(a,b);
ce1     return query(tin[a], tin[b]).second;
940 }

0dd signed main() {
5d6     winton;
45f     int q;
55a     cin >> n >> q;
c02     graph.resize(n);
f50     for (int i = 0; i < n-1; i++){
ba2         int a, b;
0a8         cin >> a >> b;
9c7         graph[--a].push_back(--b);
4c6         graph[b].push_back(a);
601     }
564     dfs(0,0);
df6     buildtable();
1f4     while(q--){
ba2         int a, b;
0a8         cin >> a >> b;
450         a--;b--;
092         int mac = lca(a,b);
389         cout << depth[a] + depth[b] - 2*depth[mac] << endl;
e88     }

1e3 }
```

3.9 Minimum Spanning Tree

```
d56 struct DSU {
8f3     vector<int> comp, sz;

c1b     DSU(int n) : comp(n), sz(n, 1) {
f7a         iota(all(comp), 0);
576     }

151     int find(int u) {
dc3         if (u == comp[u]) return u;
a07         return comp[u] = find(comp[u]);
65f     }

8bf     bool merge(int a, int b) {
605         a = find(a), b = find(b);
5c2         if (a == b) return false;
afa         sz[a] += sz[b];
```

```

ca8      comp[b] = a;
8a6      return true;
6bc    }
33f  };

63d void solve(){
14e     int n, m;
aa3     cin >> n >> m;
73b     vector<tuple<int,int,int>> edges;
370     DSU dsu(n);
8f8     edges.resize(m);
dd5     for (int i = 0; i < m; i++){
684         int a, b, w;
060         cin >> a >> b >> w;
0d4         edges[i] = {w,a,b};
261     }
5c1     sort(all(edges));
704     int cost = 0;
dd5     for(int i = 0; i < m; i++){
13d         auto [w,u,v] = edges[i];
d19         if (dsu.merge(u,v)){
45f             cost += w;
b64         }
ce9     }
16f }

```

3.10 Pontos de Articulação

```

/*
a7e Acha os articulation points do grafo
c4c */

f7d int t, c, low[MAX], pre[MAX], visited[MAX];
864 vector<int> graph[MAX], ans;

955 void dfs(int v, int p){
cda     visited[v] = 1;
98b     low[v] = pre[v] = ++t;
14e     bool art = false;
236     int filhos = 0;
284     for (auto u : graph[v]){
264         if (!visited[u]){
701             filhos++;
412             dfs(u,v);
c80             low[v] = min(low[v], low[u]);

```

```

b71         if(low[v] >= pre[v]) art = true;
4d8     }
4e6     else {
2da         if (u == p) continue;
39b         low[v] = min(low[v], pre[u]);
acd     }
f4f     }
ca1     if (v == 1 && filhos >= 2) ans.push_back(v);
021     if (v != 1 && art) ans.push_back(v);
7fd }

```

3.11 Toposort

```

// Fazer dfs pra pegar o indeg se necessario
0ad int indeg[MAX];

55b vector<int> order;
26a queue<int> q;

519 for (int i = 0; i < MAX; i++){
ac5     if (indeg[i] == 0)q.push(i);
3fd }
14d while(!q.empty()){
69f     int cur = q.front();
ba5     order.push_back(cur);
833     q.pop();
411     for (auto u : graph[cur]){
472         indeg[u]--;
f90         if (indeg[u] == 0)q.push(u);
014     }
f6a }

```

4 Matemática

4.1 Combinatoria

```

3e4 const int MOD = 1e9+7;
4e9 const int MOD = 998244353;

183 const int MAX = 1e6+7;

e4a int fat[MAX], invfat[MAX];

```

```

bf9 int fastExpo(int base, int exp) {
c92     int res = 1;
fb9     while(exp) {
8ca         if (exp & 1) res = res * base % MOD;
a6a         base = base * base % MOD;
ef0         exp >>= 1;
635     }
542     return res%MOD;
2e3 }

868 int modinv(int n) {
562     return fastExpo(n, MOD - 2);
92e }

e9d int modiv(int a, int b) {
a7c     return (a * modinv(b)) % MOD;
a02 }

0a8 void build(){
557     fat[0] = 1;
38a     for (int i = 1; i < MAX; i++) fat[i] = (fat[i-1] * i) % MOD;
507     invfat[MAX-1] = modinv(fat[MAX-1]);
e92     for (int i = MAX - 2; i >= 0; i--) invfat[i] = (invfat[i+1] *
(i+1)) % MOD;
4fa }
c52 int comb(int a, int b){
a75     if (b > a || b < 0 || a < 0) return 0;
97d     return (((fat[a] * invfat[b]) % MOD) * invfat[a-b]) % MOD;
395 }

76f int combinations(int n, int k) {
    // Basic edge cases
032     if (k < 0 || k > n) {
bb3         return 0;
edf     }
b62     if (k == 0 || k == n) {
6a5         return 1;
f14     }
dd2     if (k > n - k) {
1b5         k = n - k;
b13     }
c92     int res = 1;
64b     for (int i = 1; i <= k; ++i) {
e56         res = res * (n - i + 1) / i;
ad4     }
b50     return res;

```

```

ff6 }
//quando o denominador eh pequeno
34a int combinations_mod(int n, int k) {
e29     if (k < 0 || k > n) return 0;
bbe     if (k == 0 || k == n) return 1;
63f     if (k > n / 2) k = n - k;
bd9     int numerator = 1;
599     for(int i = 0; i < k; i++) numerator = (numerator * (n - i)) %
MOD;
cd4     int denominator = 1;
e49     for(int i = 1; i <= k; i++) denominator = (denominator * i) %
MOD;
931     return (numerator * modinv(denominator)) % MOD;
623 }

```

4.2 Cramer

```

//Calcula x e y de um sistema linear com 2 equacoes e 2 variaveis
/*
253 c1 * x + c2 * y = n
4fa c3 * x + c4 * y = m

c6d c = coeficientes

82a 5x + 3y = 35
368 2x - 8y = 12
c4c */

803 int det = (c1 * c4) - (c2 * c3);
2e5 int det_x = (n * c4) - (c2 * m);
ec2 int det_y = (c1 * m) - (n * c3);

143 int x = det_x / det;
b7f int y = det_y / det;

//mudar pra double e checar se det != 0

```

4.3 Divisores

```

5f1 for (int i = 1; i * i <= tot; ++i) {
b32     if (tot % i == 0) {
025         divisors.push_back(i);
d73         if (i * i != tot) divisors.push_back(tot / i);
7b3     }

```

4.4 EV Máximo

```
/*
799 (r/m)^n - ((r-1)/m)^n = probabilidade de ter exatamente r como o
    maior resultado
b85 somatorio de P(r) * r = EV do maximo

e08 n: numero de eventos
9cb r: resultado
0ee m: quantidade de resultados possiveis para um evento
c4c */

0e8 r * ((fastExpo((r/m), n)) - (fastExpo((ld)(r-1)/m, n)));
```

4.5 Exponenciação Rápida

```
bf9 int fastExpo(int base, int exp) {
c92     int res = 1;
fb9     while(exp) {
8ca         if (exp & 1) res = res * base % MOD;
a6a         base = base * base % MOD;
ef0         exp >>= 1;
635     }
542     return res%MOD;
2e3 }

e9d int modiv(int a, int b){
659     return (((a % MOD) * (fastExpo(b, MOD-2) % MOD)) % MOD);
e50 }

bf9 int fastExpo(int base, int exp) {
c92     int res = 1;
fb9     while(exp) {
13e         if (exp & 1) res = res * base;
6c1         base = base * base;
ef0         exp >>= 1;
319     }
b50     return res;
4fe }

83b int fastExpo(int base, int exp, int m) {
c92     int res = 1;
054     base %= m;
fb9     while(exp) {
970         if (exp & 1) res = res * base % m;
103         base = base * base % m;
```

```
ef0         exp >>= 1;
268     }
17a     return res%m;
1b8 }

ff4 ld fastExpo(ld base, int exp) {
9d0     ld res = 1;
fb9     while(exp) {
13e         if (exp & 1) res = res * base;
6c1         base = base * base;
ef0         exp >>= 1;
319     }
b50     return res;
b1b }
```

4.6 Fatoração Prima

```
4ae vector<int> factor(int n) {
fd3     vector<int> primes;
d91     for (int p = 2; p * p <= n; ++p) {
03e         while (n % p == 0) {
bce             primes.push_back(p);
f4a             n /= p;
6e7         }
4ac     }
f3d     if (n > 1)
ccc         primes.push_back(n);
a20     return primes;
8de }
```

4.7 GCD e LCM

```
ba6 int gcd(int a, int b) {
d65     return b == 0 ? a : gcd(b, a % b);
4c5 }

3ec int mmc(int a, int b) {
8b8     return a / gcd(a, b) * b;
4c5 }
```

4.8 Phi Function

```
//quantos numeros sao coprimos com N de 1 ate N
633 int phi[MAX];
```

```
0ae void precomputePHI(){
104     for (int i = 0; i < MAX; i++)
ccf         phi[i] = i;

8db     for (int i = 2; i < MAX; i++) {
e32         if (phi[i] == i) {
d87             for (int j = i; j < MAX; j += i)
a9b                 phi[j] -= phi[j] / i;
0cc         }
f9d     }
159 }
```

4.9 Segmented Sieve

```
576 vector<char> segmentedSieve(long long L, long long R) {
    // generate all primes up to sqrt(R)
4b2     long long lim = sqrt(R);
537     vector<char> mark(lim + 1, false);
d2d     vector<long long> primes;

b74     for (long long i = 2; i <= lim; ++i) {
c9d         if (!mark[i]) {
d38             primes.emplace_back(i);
1e8             for (long long j = i * i; j <= lim; j += i)
148                 mark[j] = true;
181         }
920     }

f9d     vector<char> isPrime(R - L + 1, true);
867     for (long long i : primes){
f8c         for (long long j = max(i * i, (L + i - 1) / i * i); j <=
R; j += i){
0d5             isPrime[j - L] = false;
cd3         }
24a     }
86a     if (L == 1) isPrime[0] = false;

    //para pegar os primos basta fazer i + L no vetor de isprime
3c8     return isPrime;
a43 }
```

4.10 Sieve

```
fd3 vector<int> primes;

8e0 void sieve() {
a94     vector<bool> is_prime(MAX + 1, true);
428     is_prime[0] = false;
8fc     is_prime[1] = false;
31f     for (int p = 2; p * p <= MAX; ++p) {
29c         if (is_prime[p]) {
305             for (int multiple = p * p; multiple <= MAX; multiple
+= p) {
4f8                 is_prime[multiple] = false;
16d             }
354         }
d56     }
e65     for (int i = 2; i <= MAX; ++i) {
4a3         if (is_prime[i]) {
e74             primes.push_back(i);
34c         }
b9d     }
ee0 }
```

4.11 Smallest Prime Factor

```
b32 int n, spf[MAX];

0a8 void build(){
ce4     for (int i = 2; i < MAX; i+=2) spf[i] = 2;
7b9     for (int i = 3; i < MAX; i+=2){
e8b         if (spf[i] == 0){
cdc             spf[i] = i;
37e             for (int j = i; j*i < MAX; j+=2){
9fe                 if (spf[i*j] == 0) spf[i*j] = i;
2bf             }
859         }
8e9     }
5ce }

826 vector<int> factorize(int x){
fd3     vector<int> primes;
40e     while (x > 1){
80a         int p = spf[x];
bce         primes.push_back(p);
15a         while(x%p == 0) x/=p;
```

```

64d     }

a20     return primes;
0f8 }

//Numero de fatores primos diferentes que fatoram cada numero
11f int spf[MAX], pf[MAX];

0a8 void build(){
81f     spf[1] = 1;
ce4     for (int i = 2; i < MAX; i+=2) spf[i] = 2;
7b9     for (int i = 3; i < MAX; i+=2){
e8b         if (spf[i] == 0){
cdc             spf[i] = i;
37e             for (int j = i; j*i < MAX; j+=2){
9fe                 if (spf[i*j] == 0) spf[i*j] = i;
2bf             }
859         }
8e9     }
c29 }

da9 void buildspf(){
8db     for (int i = 2; i < MAX; i++){
e09         pf[i] = pf[i/spf[i]];
a28         if (spf[i/spf[i]] != spf[i]) pf[i]++;
3a0     }
d85 }

```

5 Primitivas

5.1 Aritmética Modular

```

429 template<int p> struct mod_int {
007     int expo(int b, int e) {
856         int ret = 1;
c87         while (e) {
cad             if (e % 2) ret = ret * b % p;
9d2             e /= 2, b = b * b % p;
c42         }
edf         return ret;
170     }
572     int inv(int b) { return expo(b, p-2); }

4d7     using m = mod_int;

```

```

d93     int v;
fe0     mod_int() : v(0) {}
cde     mod_int(int v_) {
019         if (v_ >= p or v_ <= -p) v_ %= p;
bc6         if (v_ < 0) v_ += p;
2e7         v = v_;
e1d     }
74d     m& operator +=(const m& a) {
2fd         v += a.v;
ba5         if (v >= p) v -= p;
357         return *this;
c8b     }
eff     m& operator -=(const m& a) {
8b4         v -= a.v;
cc8         if (v < 0) v += p;
357         return *this;
f8d     }
4c4     m& operator *=(const m& a) {
830         v = v * int(a.v) % p; //tirar o int quando for usar define
int long long
357         return *this;
623     }
3f9     m& operator /=(const m& a) {
5d6         v = v * inv(a.v) % p;
357         return *this;
62d     }
d65     m operator -(){ return m(-v); }
e8f     m& operator ^=(int e) {
06d         if (e < 0) {
6e2             v = inv(v);
00c             e = -e;
275         }
284         v = expo(v, e);
// possivel otimizacao:
// cuidado com 0^0
// v = expo(v, e%(p-1));
357         return *this;
b6d     }
423     bool operator ==(const m& a) { return v == a.v; }
69f     bool operator !=(const m& a) { return v != a.v; }

1c6     friend istream& operator >>(istream& in, m& a) {
9bf         int val; in >> val;
d48         a = m(val);
091         return in;
a6f     }
44f     friend ostream& operator <<(ostream& out, m a) {

```

```

5a0         return out << a.v;
214     }
399     friend m operator +(m a, m b) { return a += b; }
f9e     friend m operator -(m a, m b) { return a -= b; }
9c1     friend m operator *(m a, m b) { return a *= b; }
51b     friend m operator /(m a, m b) { return a /= b; }
937     friend m operator ^(m a, int e) { return a ^= e; }
1a1 };

b49 typedef mod_int<(int)MOD> mint;

e9d int modiv(int a, int b){
659     return (((a % MOD) * (fastExpo(b, MOD-2) % MOD)) % MOD);
e50 }

```

5.2 Geometria Double

```

c83 typedef double ld;
e3b const ld DINF = 1e18;
43a const ld pi = acos(-1.0);
107 const ld eps = 1e-9;

b32 #define sq(x) ((x)*(x))

d97 bool eq(ld a, ld b) {
fe4     return fabs(a - b) <= eps;
dac }

be5 struct point { // ponto
c1e     ld x, y;
018     point(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
be2     bool operator < (const point p) const {
059         if (!eq(x, p.x)) return x < p.x;
f98         if (!eq(y, p.y)) return y < p.y;
bb3         return 0;
ab8     }
418     bool operator == (const point p) const {
ed0         return eq(x, p.x) and eq(y, p.y);
4a1     }
460     point operator + (const point p) const { return point(x+p.x,
y+p.y); }
a2f     point operator - (const point p) const { return point(x-p.x,
y-p.y); }
807     point operator * (const ld c) const { return point(x*c , y*c
); }

```

```

669     point operator / (const ld c) const { return point(x/c , y/c
); }
986     ld operator * (const point p) const { return x*p.x + y*p.y;
} // dot
d94     ld operator ^ (const point p) const { return x*p.y - y*p.x;
} // cross
b2b     friend istream& operator >> (istream& in, point& p) {
e37         return in >> p.x >> p.y;
d70     }
f70     friend ostream& operator << (ostream& os, const point& p) {
d80         return os << "(" << p.x << ", " << p.y << ")";
322     }
e09 };

b3a struct line { // reta
b1f     point p, q;
0d6     line() {}
50a     line(point p_, point q_) : p(p_), q(q_) {}
8d7     friend istream& operator >> (istream& in, line& r) {
4cb         return in >> r.p >> r.q;
858     }
ebb };

ee4 ld dist(point p, point q) { // distancia
5f3     return hypot(p.y - q.y, p.x - q.x);
23c }

06b ld dist2(point p, point q) { // quadrado da distancia
f24     return sq(p.x - q.x) + sq(p.y - q.y);
2ef }

f5c ld norm(point v) { // norma do vetor
923     return dist(point(0, 0), v);
568 }

0cf ld angle(point v) { // angulo do vetor com o eixo x
587     ld ang = atan2(v.y, v.x);
6f8     if (ang < 0) ang += 2*pi;
19c     return ang;
8c4 }

3ca ld sarea(point p, point q, point r) { // area com sinal
606     return ((q-p)^(r-q))/2;
5b6 }

6df bool col(point p, point q, point r) { // se p, q e r sao colin.
e7d     return eq(sarea(p, q, r), 0);

```

```

20f }

c65 bool ccw(point p, point q, point r) { // se p, q, r sao ccw
fa7     return sarea(p, q, r) > eps;
20e }

4cf point rotate(point p, ld th) { // rotaciona o ponto th radianos
4f9     return point(p.x * cos(th) - p.y * sin(th), p.x * sin(th) +
p.y * cos(th));
828 }

5cf point rotate90(point p) { // rotaciona 90 graus
128     return point(-p.y, p.x);
c3b }

// retorna t tal que t*v pertence a reta r
849 ld get_t(point a, line r) {
ac5     return (r.p^r.q) / ((r.p-r.q)^a);
0cf }

edc bool isvert(line r) { // se r eh vertical
87d     return eq(r.p.x, r.q.x);
0fb }

9d8 point proj(point p, line r) { // projecao do ponto p na reta r
bea     if (r.p == r.q) return r.p;
97a     r.q = r.q - r.p; p = p - r.p;
ead     point proj = r.q * ((p*r.q) / (r.q*r.q));
2cd     return proj + r.p;
d37 }

5d2 bool isinseg(point p, line r) { // se p pertence ao seg de r
517     point a = r.p - p, b = r.q - p;
b04     return eq((a ^ b), 0) and (a * b) < eps;
da8 }

956 point inter(line r, line s) { // r inter s
b99     if (eq((r.p - r.q) ^ (s.p - s.q), 0)) return point(DINF, DINF);
205     r.q = r.q - r.p, s.p = s.p - r.p, s.q = s.q - r.p;
543     return r.q * get_t(r.q, s) + r.p;
6c0 }

676 bool interseg(line r, line s) { // se o seg de r intersecta o seg
de s
2b6     if (isinseg(r.p, s) or isinseg(r.q, s) or isinseg(s.p, r) or
isinseg(s.q, r)) return 1;
085     return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) and ccw(s.p,
s.q, r.p) != ccw(s.p, s.q, r.q);
359 }

9f2 ld linepoint(point p, line r) { // distancia do ponto a reta
89a     return 2 * abs(sarea(p, r.p, r.q)) / dist(r.p, r.q);
389 }

639 ld segpoint(line r, point p) { // distancia do ponto ao seg
73d     if ((r.q - r.p)*(p - r.p) < 0) return dist(r.p, p);
951     if ((r.p - r.q)*(p - r.q) < 0) return dist(r.q, p);
199     return linepoint(p, r);
e86 }

ccc ld segseg(point a, point b, point c, point d) {
102     line ab(a,b);
001     line cd(c,d);
05c     if (interseg(ab, cd)) return 0;
494     return min({segpoint(ab,c), segpoint(ab,d), segpoint(cd,a),
segpoint(cd,b)});
3d6 }

//angulo interno entre duas linhas, mid e o ponto de intersecao entre
elas
b29 ld inner_angle(point p, point q, point mid){
ac0     point p1 = (p-mid);
325     point p2 = (q-mid);
7a8     ld rad = atan2(abs(p1 ^ p2), p1 * p2); //angulo em rad
370     ld deg = rad* 180.0 / M_PI;
816     return deg;
7ec }

//linha que corta o segmento AB no meio perpendicularmente
a8a line mediatriz(point p, point q) {
da4     point mid = (p + q) / 2.0;
b5e     point vec = q - p;
5e7     point p_vec = rotate90(vec);
b03     return line(mid, mid + p_vec);
5b8 }

//bissetriz interna e externa
//bissetriz = linha que forma angulos iguais com outras duas linhas
28b pair<line, line> bissetriz(line r, line s) {
255     if (eq((r.p - r.q) ^ (s.p - s.q), 0)) return {};
ba5     point i = inter(r,s);
1d9     point v1 = r.p - i;
4c0     point v2 = s.p - i;
27e     if (norm(v1) < eps) v1 = r.q - i;

```

```

672     if (norm(v2) < eps) v2 = s.q - i;
44a     v1 = v1 / norm(v1); v2 = v2 / norm(v2);
675     line interna = line(i, i + (v1 + v2));
8a6     line externa = line(i, i + (v1 - v2));
7fa     return {interna, externa};
d22 }

a0a point findcircle(point p, point q, point r){//acha o centro do
    circulo que encosta nos 3 pontos
55f     if (col(p,q,r)) return point(DINF,DINF);
9b6     line mid1 = mediatriz(p,q);
f9a     line mid2 = mediatriz(q,r);
294     return inter(mid1, mid2);
d41 }

a39 ld polarea(vector<point> v){ // area do poligono
9c5     ld ret = 0;
f14     for (int i = 0; i < v.size(); i++){
d0f         ret += sarea(point(0, 0), v[i], v[(i + 1) % v.size()]);
8cb     }
d03     return abs(ret);
22a }

// se o ponto ta dentro do poligono: retorna 0 se ta fora,
// 1 se ta no interior e 2 se ta na borda
fb6 int inpol(vector<point>& v, point p) { // 0(n)
8de     int qt = 0;
f14     for (int i = 0; i < v.size(); i++) {
bda         if (p == v[i]) return 2;
6af         int j = (i+1)%v.size();
e38         if (eq(p.y, v[i].y) and eq(p.y, v[j].y)) {
97f             if ((v[i]-p)*(v[j]-p) < eps) return 2;
5e2             continue;
48b         }
388         bool baixo = v[i].y+eps < p.y;
464         if (baixo == (v[j].y+eps < p.y)) continue;
366         auto t = (p-v[i])^(v[j]-v[i]);
1b4         if (eq(t, 0)) return 2;
839         if (baixo == (t > eps)) qt += baixo ? 1 : -1;
d13     }
b84     return qt != 0;
1c7 }

603 ld angle(const point &p, const point &ref) { // angulo do ponto p
    em relacao ao ponto ref
f5b     point v = p - ref;
587     ld ang = atan2(v.y, v.x);

```

```

6f8     if (ang < 0) ang += 2*pi;
19c     return ang;
220 }

//ordena os pontos de acordo com o angulo
34e void polarSort(vector<point> &v, point &ref) {
0f8     sort(v.begin(), v.end(), [&ref](point a, point b) {
9dd         return angle(a, ref) < angle(b, ref);
17a     });
060 }

971 vector<point> convex_hull(vector<point> v) { // convex hull - 0(n
    log(n))
fca     sort(v.begin(), v.end());
d76     v.erase(unique(v.begin(), v.end()), v.end());
52d     if (v.size() <= 1) return v;
ce1     vector<point> l, u;
f14     for (int i = 0; i < v.size(); i++) {
fb2         while (l.size() > 1 and !ccw(l.end()[-2], l.end()[-1],
            v[i]))
364             l.pop_back();
c35         l.push_back(v[i]);
58e     }
3e9     for (int i = v.size() - 1; i >= 0; i--) {
f19         while (u.size() > 1 and !ccw(u.end()[-2], u.end()[-1],
            v[i]))
7a8             u.pop_back();
a95         u.push_back(v[i]);
0b8     }
cfc     l.pop_back(); u.pop_back();
2e4     for (point i : u) l.push_back(i);
792     return l;
9e5 }

//comprimento do arco dado o raio e o angulo
//double comprimento = (graus * M_PI * raio) / 180.0;
//double comprimento = rad * raio;

```

5.3 Geometria Int

```

b2a struct pt {
e91     int x, y;
df1     pt(int x_ = 0, int y_ = 0) : x(x_), y(y_) {}

5bc     bool operator < (const pt p) const {
95a         if (x != p.x) return x < p.x;

```

```

89c         return y < p.y;
dcd     }

a83     bool operator == (const pt p) const {
d74         return x == p.x and y == p.y;
7b4     }

7af     pt operator + (const pt& p) const { return pt(x + p.x, y +
p.y); }
b23     pt operator - (const pt& p) const { return pt(x - p.x, y -
p.y); }
477     template<typename U>
ff5     pt operator / (const U c) const { return pt(x / c, y / c); }
477     template<typename U>
60f     pt operator * (const U c) const { return pt(x * c, y * c); }
abb     int operator * (const pt& p) const { return x * p.x + y * p.y;
}
476     int operator ^ (const pt& p) const { return x * p.y - y * p.x;
}

e45     friend istream& operator >> (istream& in, pt& p) { return in
>> p.x >> p.y; }
03c     friend ostream& operator << (ostream& os, const pt& p) {
return os << "(" << p.x << ", " << p.y << ")"; }
c58 };

b3a struct line {
730     pt p, q;
0d6     line() {}
4b8     line(pt p_, pt q_) : p(p_), q(q_) {}

8d7     friend istream& operator >> (istream& in, line& r) {
4cb         return in >> r.p >> r.q;
858     }

6b7     friend ostream& operator << (ostream& os, line& r) {
e8e         return os << "[" << r.p << " -> " << r.q << "]";
bbb     }
3b6 };

46d int sq(pt p){
8a7     return p.x*p.x + p.y*p.y;
b0e }

//checa se dois vetores sao perpendiculares
6b8 bool perpendicular(pt v, pt w){
350     return (v*w == 0);

```

```

99e }

//ccw, 0 colineares, -1 r a direita, 1 r a esquerda
f2b int orientation(pt p, pt q, pt r){
c5b     int val = ((q-p) ^ (r-p));
db6     if (!val) return 0;
326     return (val > 0) ? 1 : -1;
dd6 }

//checa se um ponto esta contido no angulo do segmento pq e pr
e82 bool onAngle (pt p, pt q, pt r, pt g){
999     if (!orientation(p,q,r)) return false;
673     if (orientation(p,q,r) > 0) swap(q,r);
d46     return orientation(p,q,g) >= 0 && orientation(p,r,g) <= 0;
edb }

// quadrante de um ponto
c31 int quad(pt p) {
dbb     return (p.x<0)^3*(p.y<0);
fcf }

//dist quadrada de dois pontos
9ea int dist2(pt p, pt q){
fda     return sq(p - q);
fb2 }

// 2 * area com sinal
a15 int sarea2(pt p, pt q, pt r) {
586     return (q-p)^(r-q);
9ef }

// se p, q e r sao colin.
e32 bool col(pt p, pt q, pt r) {
034     return sarea2(p, q, r) == 0;
a08 }

// 2 * area do poligono
fc1 int polarea2(vector<pt> v) {
7c9     int ret = 0;
c6e     for (int i = 0; i < v.size(); i++)
532         ret += sarea2(pt(0, 0), v[i], v[(i + 1) % v.size()]);
d03     return abs(ret);
87e }

//checa se o poligono e convexo
f12 bool isConvex(vector<pt> p) {
ff7     bool hasPos = false, hasNeg = false;

```

```

ed0     for (int i = 0, n = p.size(); i < n; i++) {
568         int o = orientation(p[i], p[(i+1)%n], p[(i+2)%n]);
f42         if (o > 0) hasPos = true;
162         if (o < 0) hasNeg = true;
455     }
f61     return !(hasPos && hasNeg);
61d }

ae2 bool half(pt p) { // true if in blue half
184     assert(p.x != 0 || p.y != 0); // the argument of (0,0)
        isundefined
a12     return p.y > 0 || (p.y == 0 && p.x < 0);
a70 }

//ordena os pontos de acordo com o angulo
61f void polarSort(vector<pt> &v) {
2f2     sort(v.begin(), v.end(), [](pt v, pt w) {
8ba         return make_tuple(half(v), 0, sq(v)) <
99d             make_tuple(half(w), (v~w), sq(w));
486     });
34d }

//rotaciona o vetor 90 graus
ab1 pt rotate90(pt p) {
a0d     return pt(-p.y, p.x);
e4a }

//retorna um ponto que esta no centro entre outros dois pontos
143 pt meio(pt p, pt q){
5c9     return (p+q)/2;
603 }

//retorna uma linha perpendicular a linha s que passa pelo ponto p
9c9 line perpthrough(line s, pt p){
2d4     return {p, p + rotate90(s.p - s.q)};
299 }

// se p esta na linha de r
985 bool onLine(line r, pt p){
489     return col(r.p, r.q, p);
        // return ((r.q - r.p) ^ (p - r.p)) == 0;
ea4 }

// se p pertence ao seg de r
63e bool isinseg (line r, pt p) {
eb7     pt a = r.p - p;
e99     pt b = r.q - p;

```

```

2ac     return (a ^ b) == 0 and (a * b) <= 0;
336 }

147 bool intersect(line r, line s){
0d5     int o1 = orientation(r.p, r.q, s.p);
f92     int o2 = orientation(r.p, r.q, s.q);
033     int o3 = orientation(s.p, s.q, r.p);
885     int o4 = orientation(s.p, s.q, r.q);

bb3     if (o1 != o2 && o3 != o4) return true;

171     if (o1 == 0 && isinseg(r, s.p)) return true;
0f8     if (o2 == 0 && isinseg(r, s.q)) return true;
082     if (o3 == 0 && isinseg(s, r.p)) return true;
718     if (o4 == 0 && isinseg(s, r.q)) return true;

dif     return false;
e86 }

// numero de pontos inteiros no segmento
fae int segpts(line r) {
508     return 1 + gcd(abs(r.p.x - r.q.x), abs(r.p.y - r.q.y));
28a }

// se o ponto ta dentro do poligono: retorna 0 se ta fora,
// 1 se ta no interior e 2 se ta na borda
8e7 int inpol(vector<pt>& v, pt p){ // 0(n)
8de     int qt = 0;
f14     for (int i = 0; i < v.size(); i++) {
bda         if (p == v[i]) return 2;
6af         int j = (i+1)%v.size();
cc6         if (p.y == v[i].y and p.y == v[j].y) {
547             if ((v[i]-p)*(v[j]-p) <= 0) return 2;
5e2             continue;
b47         }
78c         bool baixo = v[i].y < p.y;
057         if (baixo == (v[j].y < p.y)) continue;
366         auto t = (p-v[i])^(v[j]-v[i]);
2ad         if (!t) return 2;
0bb         if (baixo == (t > 0)) qt += baixo ? 1 : -1;
9cf     }
b84     return qt != 0;
afd }

9dc vector<pt> convex(vector<pt> v){
304     sort(all(v));
64f     v.erase(unique(all(v)), v.end());

```

```

/*
31d     IMPORTANTE! em casos que pede o convex hull incluindo pontos
colineares, mudar o ccw para >=
c4c     */
52d     if(v.size() <= 1) return v;
526     vector<pt> l, u;
f14     for(int i = 0; i < v.size(); i++){
e35         while(l.size() > 1 and !ccw(l.end()[-2], l.end()[-1],
v[i])){
364             l.pop_back();
b47         }
c35         l.push_back(v[i]);
189     }
3e9     for(int i = v.size() - 1; i >= 0; i--){
0c7         while(u.size() > 1 and !ccw(u.end()[-2], u.end()[-1],
v[i])){
7a8             u.pop_back();
9bd         }
a95         u.push_back(v[i]);
625     }
364     l.pop_back();
7a8     u.pop_back();
82b     for(pt i : u) l.push_back(i);
792     return l;
4ae }

```

5.4 Matrix Exponentiation

```

14e     int n, m;

df8     matrix(int n_, int m_, bool ident = false) :
vector<vector<int>>>(n_, vector<int>(m_, 0)), n(n_), m(m_) {
94e         if (ident) {
df7             assert(n == m);
a89             for (int i = 0; i < n; i++) (*this)[i][i] = 1;
359         }
456     }

483     matrix(const vector<vector<int>>& c) : vector<vector<int>>(c),
a3d     n(c.size()), m(c[0].size()) {}

d01     matrix operator*(const matrix &a) const {
eda         assert(m == a.n);
ca0         matrix res(n, a.m);
603         for (int i = 0; i < n; i++){
cf4             for (int j = 0; j < a.m; j++){

```

```

917             for (int k = 0; k < m; k++){
956                 res[i][j] = (res[i][j] + ((*this)[i][k] *
a[k][j]) % MOD) % MOD;
368             }
830         }
a00     }
b50     return res;
3b4 }

3d2     matrix operator^(int e) const {
cd2         matrix res(n,n,true);
2de         matrix base = *this;
c87         while (e) {
d4a             if (e&1) res = res * base;
6c1             base = base * base;
cc2             e >>= 1;
ece         }
b50         return res;
81d     }
214 };

```

6 Problemas

6.1 3 equacoes 4 variaveis

```

// e(s) = (y+p-r)/2 - s
// w(s) = (y-p-r)/2 - s
// s(s) = s

// n(s) = a + s
// e(s) = b - s
// w(s) = c - s
// s(s) = s

// M = max{(|a+s|), (|b-s|), (|c-s|), (|s|)}

// |a+s| <= M    ->    -M-a <= s <= M-a
// |b-s| <= M    ->    -M+b <= s <= M+b
// |c-s| <= M    ->    -M+c <= s <= M+c
// |s| <= M      ->    -M <= s <= M

// achar o minimo e maximo do intervalo de s

// smin = max(-a,b,c,0)

```

```

// smax = max(-a,b,c,0)
Odd signed main(){
45f     int q;
11b     cin >> q;
1f4     while(q--){
172         ld p, r, y;
b47         cin >> p >> r >> y;
bf3         ld a = r;
09e         ld b = (y+p-r)/2;
305         ld c = (y-p-r)/2;

c79         ld smin = max({-a,b,c,(ld)0.0});
34f         ld smax = min({-a,b,c,(ld)0.0});

e2e         ld s = (smin+smax)/2;

4cb         ld n = r+s;
e48         ld e = b-s;
c2d         ld w = c-s;

e8a         cout << fixed << setprecision(7) << n << ' ' << e << ' '
<< s << ' ' << w << ' ' << endl;
bb1     }
cb3 }

```

6.2 Achar prefixos que formam a string toda

```

878 mt19937 rng((int)
    chrono::steady_clock::now().time_since_epoch().count());

463 int uniform(int l, int r) {
a7f     uniform_int_distribution<int> uid(l, r);
f54     return uid(rng);
d9e }

9e0 template<int MOD> struct str_hash {
c63     static int P;
648     vector<int> h, p;
ea8     str_hash(string s) : h(s.size()), p(s.size()) {
7a2         p[0] = 1, h[0] = s[0];
ad7         for (int i = 1; i < s.size(); i++)
84c             p[i] = p[i - 1]*P%MOD, h[i] = (h[i - 1]*P + s[i])%MOD;
1ef     }
6cf     int operator()(int l, int r) { // retorna hash s[l...r]
796         int hash = h[r] - (l ? h[l - 1]*p[r - l + 1]%MOD : 0);

```

```

dfd         return hash < 0 ? hash + MOD : hash;
e8b     }
6cf };
217 template<int MOD> int str_hash<MOD>::P = uniform(256, MOD - 1); //
    1 > |sigma|

63d void solve() {
21a     string s; cin >> s;
556     str_hash<MOD> hasher(s);
163     int n = s.size();
78a     for(int i = 1; i <= n; i++){
bdd         if(i == n) cout << n << endl;
4e6         else{
9f5             int h1 = hasher(0, n-i-1);
576             int h2 = hasher(i, n-1);
18b             if(h1 == h2){
bfe                 cout << i << " ";
ef7             }
d8b         }
e54     }
fb3 }

```

6.3 Artwork

```

//Grafo de varios circulos
//saber se chega do 00 ao nm sem encostar nos circulos

add int n, m, k;
d56 struct DSU{
1a8     int n;
632     vector<int> p;
02d     vector<int> r;
a95     vector<vector<int>> encosta;
2ea     DSU(int _n, vector<vector<int>> _marcador){
8fd         n = _n;
1c4         p.resize(n);
7e4         r.resize(n);
972         encosta = _marcador;
603         for(int i = 0; i < n; i++){
f13             p[i] = i;
fd1             r[i] = 0;
606         }
68f     }
57b     int find(int x){
eab         return (p[x] == x? x : p[x] = find(p[x]));

```

```

ab9     }
440     void unite(int a, int b){
605         a = find(a), b = find(b);
d54         if(a == b) return;
dac         if(r[a] == r[b]) r[a]++;
d1b         if(r[a] > r[b]){
67e             r[a] += r[b];
264             p[b] = a;
1cd             for(int i = 0; i < 4; i++){
9e8                 encosta[a][i] |= encosta[b][i];
31b             }
e6c         }else{
2e3             r[b] += r[a];
84b             p[a] = b;
1cd             for(int i = 0; i < 4; i++){
7a5                 encosta[b][i] |= encosta[a][i];
f4c             }
701         }
15f     };
b43 };
80d int dist(int x1, int y1, int x2, int y2){
a9d     return (x1-x2)*(x1-x2) + (y1-y2)*(y1-y2);
fb0 }
0dd signed main(){
9ae     fastio;
5fe     cin >> n >> m >> k;
6ef     int pos = 1;
c56     vector<tuple<int, int, int>> v;
0bd     vector<vector<int>> marcador(k, vector<int>(4, 0));
b3d     for(int i = 0; i < k; i++){
32e         int a, b, c; cin >> a >> b >> c;
6c9         v.push_back({a, b, c});
ea0         if(a-c <= 0) marcador[i][0] = 1;
8a1         if(a+c >= n) marcador[i][1] = 1;
552         if(b-c <= 0) marcador[i][2] = 1;
dfd         if(b+c >= m) marcador[i][3] = 1;
f3a     }

313     DSU dsu(k, marcador);
b3d     for(int i = 0; i < k; i++){
dde         auto [x1, y1, r1] = v[i];
453         for(int j = i+1; j < k; j++){
b4a             if(j == i) continue;
f7c             auto [x2, y2, r2] = v[j];
d6c             if(dist(x1, y1, x2, y2) <= (r1 + r2)*(r1+r2)){
7fd                 dsu.unite(i, j);
823         }

```

```

88d     }
c50     }

b3d     for(int i = 0; i < k; i++){
a44         auto p = dsu.find(i);
00d         if(dsu.encosta[p][0] && dsu.encosta[p][1]){
792             pos = 0;
c2b             break;
873         }if(dsu.encosta[p][2] && dsu.encosta[p][3]){
792             pos = 0;
c2b             break;
fd0         }
089         if(dsu.encosta[p][0] && dsu.encosta[p][2]){
792             pos = 0;
c2b             break;
3c1         }
b6e         if(dsu.encosta[p][1] && dsu.encosta[p][3]){
792             pos = 0;
c2b             break;
aff         }
035     }
c69     if(pos) cout << "S\n";
100     else cout << "N\n";
77c }

```

6.4 Centro de Árvore

```

309 vector<vector<int>> graph;
ac8 int n, dist[3][MAX];

69f void dfs(int v, int p, int id){
284     for (auto u : graph[v]){
2ab         if (u == p || dist[id][u] > 0) continue;
14a         dist[id][u] = dist[id][v] + 1;
041         dfs(u,v,id);
0ea     }
d8f }

da9 int distance(int id){
bda     int u = -1, mx = 0;
78a     for (int i = 1; i <= n; i++){
efa         if (dist[id][i] > mx){
af1             u = i;
87e             mx = dist[id][i];
da3         }
c2f     }
03f     return u;

```

```

f26 }
738 int cont(int v, int p, int d){
e40     if (d == 0) return 1;
b3d     int cur = 0;
284     for (auto u : graph[v]){
2da         if (u == p) continue;
72d         cur += cont(u,v,d-1);
075     }
75e     return cur;
2a4 }

0dd signed main () {
5d6     winton;
a68     cin >> n;
8f4     graph.resize(n+1);
6f5     for (int i = 1; i < n; i++){
ba2         int a, b;
0a8         cin >> a >> b;
8fa         graph[a].push_back(b);
4c6         graph[b].push_back(a);
080     }
c7e     dfs(1,0,0);
398     int a = distance(0);
f96     dfs(a,0,1);
924     int b = distance(1);
ee1     dfs(b,0,2);

ade     vector<int> center;
78a     for (int i = 1; i <= n; i++){
8b2         if (dist[1][i] + dist[2][i] == dist[1][b] &&
abs(dist[1][i] - dist[2][i]) <= 1){
1dc             center.push_back(i);
cef         }
7f6     }
319 }

```

6.5 Coin Collector

```

4d3 vector<vector<int>> graph, rgraph;
c7b vector<set<int>> gcomp;
7b4 stack<int> st;
daa int n, com, indeg[MAX], dp[MAX], coins[MAX], visited[MAX],
comp[MAX], gcoins[MAX];

76a void dfs(int v) {
cda     visited[v] = 1;

```

```

284     for (auto u : graph[v]){
e39         if (!visited[u]) dfs(u);
6e1     }
789     st.push(v);
850 }

664 void scc(int v, int c) {
cda     visited[v] = 1;
d5f     comp[v] = c;
af8     for (auto u : rgraph[v]){
157         if (!visited[u]) scc(u, c);
7bd     }
7e6 }

db8 void kosaraju() {
1ef     for (int i = 0; i < n; i++) visited[i] = 0;
76b     for (int i = 0; i < n; i++) if (!visited[i]) dfs(i);
1ef     for (int i = 0; i < n; i++) visited[i] = 0;
3c6     while (st.size()) {
f1a         int u = st.top();
25a         st.pop();
264         if (!visited[u]) {
e25             scc(u, com++);
1b3         }
338     }
4de }

0dd signed main(){
2e6     int m;
aa3     cin >> n >> m;
c02     graph.resize(n);
21d     rgraph.resize(n);

5ed     for (int i = 0; i < n; i++) cin >> coins[i];
dd5     for (int i = 0; i < m; i++){
ba2         int a, b;
0a8         cin >> a >> b;
9c7         graph[--a].push_back(--b);
8d3         rgraph[b].push_back(a);
a20     }
75d     kosaraju();
b2b     gcomp.resize(com);
778     for(int v = 0; v < n; v++){
284         for (auto u : graph[v]){
aa8             if (comp[u] == comp[v]) continue;

ad6             if (gcomp[comp[v]].insert(comp[u]).second){

```

```

980         indeg[comp[u]]++;
384     }
504 }
601 }
7c8 for(int i = 0; i < n; i++) gcoins[comp[i]] += coins[i];
425 for(int i = 0; i < n; i++) dp[i] = gcoins[i];
603 for(int i = 0; i < n; i++) {
ad3     debug(i, comp[i], gcoins[i], indeg[i]);
d2c }

26a queue<int> q;
c97 for (int i = 0; i < com; i++){
ac5     if (indeg[i] == 0) q.push(i);
21b }

14d while (!q.empty()){
b1e     int v = q.front();
833     q.pop();
1ed     for (auto u : gcomp[v]){
7bd         debug(u);
f27         dp[u] = max(dp[u], dp[v] + gcoins[u]);
e80         if (--indeg[u] == 0) q.push(u);
ee4     }
b12 }
1a4 int ans = 0;
603 for (int i = 0; i < n; i++){
1cf     ans = max(ans, dp[i]);
f56 }
886 cout << ans << endl;
187 }

```

6.6 Creating Multiples

```

0dd signed main(){
5d6     winton;
b56     int b, l;
110     cin >> b >> l;
e28     vector<int> d(l);
cf7     for(auto &u : d) cin >> u;
afa     int base = b+1;
660     int resto = 0;
d00     for (int i = 0; i < l; i++) {
01a         int termo = d[i];
002         if (((l-1-i)&1)) {
142             termo = -termo;
87e     }

```

```

5c5         resto = (resto+termo % base + base) % base;
de7     }
9ed     if (resto == 0) {
005         cout << "0 0" << endl;
bb3         return 0;
a7c     }
d00     for (int i = 0; i < l; i++) {
4eb         int reducao;
600         if ((l-1-i)&1) {
87d             reducao = resto;
0b4         }
2b6         else reducao = base - resto;

aa1         if (d[i] >= reducao) {
092             cout << i + 1 << " " << d[i] - reducao << endl;
bb3             return 0;
c6f         }
8da     }
f82     cout << "-1 -1" << endl;;
960 }

```

6.7 Divisor Analysis

// quantidade, soma e produto dos divisores

```

bf9 int fastExpo(int base, int exp) {
c53     int r = 1;
479     base %= MOD;
fb9     while(exp) {
727         if (exp & 1) r = r * base % MOD;
a6a         base = base * base % MOD;
ef0         exp >>= 1;
a45     }
4c1     return r;
844 }

fdd int modinverse(int n) {
562     return fastExpo(n, MOD - 2);
8ff }

e9d int modiv(int a, int b){
0e2     return (a * modinverse(b))%MOD;
656 }

0dd signed main(){
1a8     int n;

```

```

a68     cin >> n;
dd9     bool par = true;
b7a     vector<int> p(n), k(n);
f40     int cnt = 1, sum = 1, prod = 1, cnt2 = 1;
603     for (int i = 0; i < n; i++){
742         cin >> p[i] >> k[i];
9d8         if (k[i]&1) par = false;
dd3     }
603     for (int i = 0; i < n; i++){
ed6         cnt = (cnt * (k[i]+1)) % MOD;
57a         cnt2 = (cnt2 * (k[i]+1)) % (2*(MOD-1));

a4a         int exp = (k[i] % (MOD-1));
db7         exp = (exp+1) % (MOD-1);
//debug(exp);
ab0         int num = (fastExpo(p[i],exp)-1 + MOD) % MOD;
27b         int den = (p[i]-1 + MOD) % MOD;
ce9         sum = (sum * modiv(num,den)%MOD)%MOD;
a41     }
603     for (int i = 0; i < n; i++){
b5b         if (!par){
bf2             int half = (cnt2/2)%MOD;
220             int exp = (k[i]*half)%MOD;
dc0             prod = (prod * (fastExpo(p[i],exp)%MOD)) % MOD;
b1f         }
4e6         else{
c85             int exp = ((k[i]/2)*cnt2)%MOD;
dc0             prod = (prod * (fastExpo(p[i],exp)%MOD)) % MOD;
719         }
e4f     }
d2a     cout << cnt%MOD << ' ' << sum%MOD << ' ' << prod%MOD << endl;
a52 }

```

6.8 Dynamic Inversions

```

/*
115 You are given a permutation {1,2,3,...,n}. Remove m of them one
    by one, and output the number of
41d inversion pairs before each removal. The number of inversion pairs
    of an array A is the number of
a9e ordered pairs (i, j) such that i < j and A[i] > A[j].
c4c */

2b7 #include <bits/stdc++.h>
ca4 using namespace std;

```

```

613 #define all(x) x.begin(), x.end()
1c9 #define rall(x) x.rbegin(), x.rend()
42a #define endl '\n'
a7f #define int long long
d40 #define ld long double

// #define DEBUG

733 #if defined(DEBUG)
769     #define winton (void)0
c88     #define debug(...) cerr << BOLD_BLUE << "[" << __func__ << ":"
        << __LINE__ << "]" << RESET << " "; debug_out(#__VA_ARGS__,
        __VA_ARGS__)
8c1 #else
62e     #define winton
ios_base::sync_with_stdio(false), cin.tie(NULL), cout.tie(NULL)
e3e     #define debug(...) (void)0
f2e #endif

a6b template<class T = int> struct bit2d {
acf     vector<T> X;
a84     vector<vector<T>> Y, t;

709     int ub(vector<T>& v, T x) {
dde         return upper_bound(v.begin(), v.end(), x) - v.begin();
9cc     }
5cb     bit2d(vector<pair<T, T>> v) {
2e1         for (auto [x, y] : v) X.push_back(x);
fd4         sort(X.begin(), X.end());
1ee         X.erase(unique(X.begin(), X.end()), X.end());

d56         t.resize(X.size() + 1);
d12         Y.resize(t.size());
3d0         sort(v.begin(), v.end(), [](auto a, auto b) {
e8f             return a.second < b.second; });
961         for (auto [x, y] : v) for (int i = ub(X, x); i < t.size();
            i += i&-i)
b75             if (!Y[i].size() or Y[i].back() != y)
                Y[i].push_back(y);

7c7         for (int i = 0; i < t.size(); i++) t[i].resize(Y[i].size()
+ 1);
8cc     }

e78     void update(T x, T y, T v) {
2a9         for (int i = ub(X, x); i < t.size(); i += i&-i)

```

```

cd2         for (int j = ub(Y[i], y); j < t[i].size(); j += j&-j)
t[i][j] += v;
533     }

5d2     T query(T x, T y) {
966         T ans = 0;
c54         for (int i = ub(X, x); i; i -= i&-i)
4fb             for (int j = ub(Y[i], y); j; j -= j&-j) ans += t[i][j];
ba7         return ans;
62d     }
46d     T query(T x1, T y1, T x2, T y2) {
fcf         return query(x2, y2)-query(x2, y1-1)-query(x1-1,
y2)+query(x1-1, y1-1);
232     }
6a7 };

0dd signed main(){
5d6     winton;
3b9     int n, q;
83f     while(cin >> n >> q){

7ed         vector<int> a(n), pos(n+1);
a84         vector<pair<int,int>> plano(n);
603         for (int i = 0; i < n; i++){
788             cin >> a[i];
d1d             pos[a[i]] = i+1;
1b4             plano[i] = {i+1, a[i]};
df0         }

9ff         bit2d bit(plano);
2cb         for (auto [x,y] : plano) bit.update(x,y,1);
a93         int sum = 0;
e84         for (auto [x,y] : plano){
d6b             sum += bit.query(x+1, 1, n, y-1);
7db         }
cde         debug(sum);

1f4         while(q--){
// cout << "sum: " ;
d94             cout << sum << endl;

93d             int y;
e90             cin >> y;
874             int x = pos[y];
e04             bit.update(x,y,-1);
053             sum -= bit.query(1, y+1, x-1, n);
89a             sum -= bit.query(x+1, 1, n, y-1);

```

```

5b7     }
7e6     }
698 }

```

6.9 Graph Paths I

```

0dd signed main(){
add     int n, m, k;
5fe     cin >> n >> m >> k;
70d     vector<vector<int>> con(n, vector<int>(n));
dd5     for (int i = 0; i < m; i++){
ba2         int a, b;
0a8         cin >> a >> b;
67f         con[--a][--b]++;
69c     }
db1     matrix mt(con);

654     mt = mt^k;
4aa     cout << mt[0][n-1] << endl;
c21 }

```

6.10 Graph Paths I

```

// Minimum path length that visits k vertices
0dd signed main(){
add     int n, m, k;
5fe     cin >> n >> m >> k;
067     vector<vector<int>> con(n, vector<int>(n, INF));
dd5     for (int i = 0; i < m; i++){
70f         int a, b, c;
e22         cin >> a >> b >> c;
583         a--;
199         b--;
d81         con[a][b] = min(c, con[a][b]);
946     }
db1     matrix mt(con);
654     mt = mt^k;
f57     int ans = mt[0][n-1];
4bc     if (ans == INF) cout << "-1" << endl;
519     else cout << ans << endl;

00a }

```

6.11 Hungaro

```
// Resolve o problema de assignment (matriz n x n)
// Colocar os valores da matriz em 'a' (pode < 0)
// assignment() retorna um par com o valor do
// assignment minimo, e a coluna escolhida por cada linha
//
// Se precisar calcular o produtorio da pra guardar o log(double(x))
// no a[i][j]
// -log se for maior produtorio e +log se for o menor produtorio
// 0(n^3)

a6a template<typename T> struct hungarian{
1a8     int n;
a08     vector<vector<T>> a;
f36     vector<T> u, v;
5ff     vector<int> p, way;
f1e     T inf;

aea     hungarian(int _n, vector<vector<T>> _a){
8fd         n = _n;
92d         u.resize(n+1);
eac         v.resize(n+1);
181         p.resize(n+1);
c7c         way.resize(n+1);
ad6         a = _a;
d80         inf = INT_MAX;
c00     }

d67     pair<T, vector<int>> assignment() {
78a         for (int i = 1; i <= n; i++) {
8c9             p[0] = i;
625             int j0 = 0;
ce7             vector<T> minv(n+1, inf);
241             vector<int> used(n+1, 0);
016             do {
472                 used[j0] = true;
d24                 int i0 = p[j0], j1 = -1;
7e5                 T delta = inf;
9ac                 for (int j = 1; j <= n; j++) if (!used[j]) {
7bf                     T cur = a[i0-1][j-1] - u[i0] - v[j];
9f2                     if (cur < minv[j]) minv[j] = cur, way[j] = j0;
821                     if (minv[j] < delta) delta = minv[j], j1 = j;
4d1                 }
f63                 for (int j = 0; j <= n; j++)
2c5                     if (used[j]) u[p[j]] += delta, v[j] -= delta;
6ec                     else minv[j] -= delta;
```

```
6d4                 j0 = j1;
f4f             } while (p[j0] != 0);
016             do {
4c5                 int j1 = way[j0];
0d7                 p[j0] = p[j1];
6d4                 j0 = j1;
886             } while (j0);
38d         }
306         vector<int> ans(n);
6db         for (int j = 1; j <= n; j++) ans[p[j]-1] = j-1;
da3         return make_pair(-v[0], ans);
979     }
51f };
```

6.12 Intervalo da String Palíndromico

```
//Acha o maior intervalo que pode ser reordenado como palindrome numa
//string
//Pode ser adaptado para contar quantos palindromes e possivel fazer

071 int size = (1<<26);
7cf vector<int> first(size, -1);
198 int mask = 0;
b55 first[0] = 0;
87c int ans = 1;
1cf for (int i = 0; i < s.size(); i++) {
90e     int chab = s[i] - 'a';
fe3     mask ^= (1 << chab);
208     if (first[mask] != -1) ans = max(ans, (i - first[mask])+1);
033     for (int k = 0; k < 26; ++k) {
f9c         int targetmask = mask ^ (1 << k);
dc8         if (first[targetmask] != -1) ans = max(ans, (i -
first[targetmask])+1);
804     }
c26     if (first[mask] == -1) first[mask] = i+1;
b7d }
```

6.13 Inversões

```
833 int countAndMerge(vector<int>& arr, int l, int m, int r) {
fd8     int n1 = m - l + 1, n2 = r - m;
e11     vector<int> left(n1), right(n2);
bac     for (int i = 0; i < n1; i++) left[i] = arr[i + l];
d74     for (int j = 0; j < n2; j++) right[j] = arr[m + 1 + j];
```

```

5e8     int res = 0, i = 0, j = 0, k = 1;
40c     while (i < n1 && j < n2) {
839         if (left[i] <= right[j]) arr[k++] = left[i++];
4e6         else {
20c             arr[k++] = right[j++];
091             res += (n1 - i);
481         }
ceb     }
694     while (i < n1) arr[k++] = left[i++];
3cc     while (j < n2) arr[k++] = right[j++];
b50     return res;
35e }

```

```

462 int countInv(vector<int>& arr, int l, int r){
11e     int res = 0;
b14     if (l < r) {
ec9         int m = (r + l) / 2;
453         res += countInv(arr, l, m);
257         res += countInv(arr, m + 1, r);
470         res += countAndMerge(arr, l, m, r);
7e6     }
b50     return res;
7d1 }

```

```

92c int inversions(vector<int> &arr) {
851     int n = arr.size();
cd9     return countInv(arr, 0, n-1);
997 }

```

6.14 Kadane

```

559 int kadane(const & vector<int> x) {
66c     int sum = x[0];
fca     int mx = x[0];
b19     for (int i = 1; i < x.size(); i++) {
cbb         sum = max(sum + x[i], x[i]);
2f2         mx = max(mx, sum);
7b0     }
b9a     return mx;
df8 }

```

6.15 MissingCoinSUM

```

0dd signed main(){
a68     cin >> n;
2b5     for(int i = 0; i < n; i++) cin >> arr[i];
7a3     sort(arr, arr+n);

87c     int ans = 1;
603     for(int i = 0; i < n; i++){
eff         if(arr[i] > ans){
886             cout << ans << endl;
bb3             return 0;
2b9         }
e79         ans += arr[i];
54a     }

886     cout << ans << endl;
3ec }

```

6.16 Number of Pairs (GCD e LCM)

28b alguma coisa de manipular a expressao ja que o $\text{lcm}(a,b) = \frac{a*b}{\text{gcd}(a,b)}$

4e4 $x = c*(a*b/\text{gcd}(a,b)) - d*\text{gcd}(a,b)$

618 $\text{gcd}(a,b) = g$

fa2 $\text{lcm}(a,b) = g*x*y$ onde $\text{gcd}(x,y) = 1$

12c por ex se $x = 5$ e $y = 3$

3d2 $\text{lcm}(5,3) = 5*3*\text{gcd}(5,3)\{1\} = 15$

749 se a,b nao forem coprimos achamos um x,y que fazem eles serem coprimos, de maneira que

3c9 $a = 4$

82b $b = 6$

9f7 $\text{lcm}(a,b) = 12$

20a $\text{gcd}(a,b) = 2$

4f8 $\text{lcm}(a,b) = x*y*\text{gcd}(a,b)$

701 $12 = x*y*2$

12d $x*y = 6$

587 $x = 3$

983 $y = 2$

583 ou

6d1 $x = 6$

6a2 $y = 1$

```

e36 lcm(4,6) = 3*2*2 correto
64f lcm(4,6) = 6*1*2 correto

578 entao lcm(a,b) = x*y*gcd(a,b)

672 x = c*x*y*g - d*g
b83 x = g((c*x*y) - d)
856 x/g = c*x*y - d
c4c */

f09 int spf[MAX], factors[MAX];

0a8 void build(){
81f     spf[1] = 1;
ce4     for (int i = 2; i < MAX; i+=2) spf[i] = 2;
7b9     for (int i = 3; i < MAX; i+=2){
e8b         if (spf[i] == 0){
cdc             spf[i] = i;
37e             for (int j = i; j*i < MAX; j+=2){
9fe                 if (spf[i*j] == 0) spf[i*j] = i;
2bf             }
859         }
8e9     }
c29 }

cbc void buildfactors(){
8db     for (int i = 2; i < MAX; i++){
4b7         factors[i] = factors[i/spf[i]];
b6b         if (spf[i/spf[i]] != spf[i]) factors[i]++;
2ba     }
9cb }

c09 vector<int> getdiv(int x){
5ce     vector<int> d;
09a     for (int i = 1; i * i <= x; i++) {
72f         if (x % i == 0){
f9a             d.push_back(i);
331             if (i*i!=x) d.push_back(x/i);
7d0         }
a93     }
bae     sort(all(d));
be2     return d;
337 }

63d void solve(){
218     int c, d, x;

```

```

234     cin >> c >> d >> x;
0e8     vector<int> divisors = getdiv(x);
1a4     int ans = 0;
3fa     for (auto gc : divisors){
266         if ((x/gc)+d) % c != 0) continue;
ce4         int goal = (x/gc + d) / c;
//goal significa que eu quero achar dois caras que
//multiplicados sejam goal, e o gcd deles seja 1
//nisso eu vou descobrir a2 e b2, para achar a e b eu
//multiplico pelo u (gcd(a,b)) e calculo o lcm e depois
//testo os pares n sei como ainda
//aqui eu acho o lcm e consigo descobrir o gcd de a e b
//no caso o gcd ja eh meu divisor
//ai basta achar as combinacoes validas de a e b
//fatorar o goal e ver os fatores primos, para cada fator
//primo, a e b podem optar por receber ou nao ele
//todos os fatores primos precisam ser usados, ou em a ou
//em b
//entao temos como resposta a adicao de todas as
//combinacoes de k (qtd de fatores primos) em a,b
6fc         int k = factors[goal];
//debug(k);
741         ans += (1<<k);
495     }
886     cout << ans << endl;
450 }

0dd signed main(){
5d6     winton;
6f2     build();
d37     buildfactors();
8bd     int t;
f1b     cin >> t;
6e6     while(t--) solve();
af8 }

```

6.17 Path Queries 2

```

2b7 #include <bits/stdc++.h>
ca4 using namespace std;

b63 const int MAX = 2e5+7;

fb1 namespace SegTree {
1a8     int n;
8f8     int pot2;

```

```

562     vector<int> tree;

0d8     void build(int n2, int* v2) {
1e3         n = n2;
fa5         pot2 = 1;
2c6         while (pot2 < n) pot2 <= 1;
26a         tree.assign(2 * pot2, 0);
4e2         for (int i = 0; i < n; ++i) tree[pot2 + i] = v2[i];
813         for (int i = pot2 - 1; i >= 1; --i) tree[i] =
max(tree[2*i], tree[2*i+1]);
641     }

4ea     int query(int a, int b) {
026         if (a > b) return 0;
238         a += pot2; b += pot2;
11e         int res = 0;
cce         while (a <= b) {
16c             if (a & 1) res = max(res, tree[a++]);
410             if (!(b & 1)) res = max(res, tree[b--]);
691             a >>= 1; b >>= 1;
80e         }
b50         return res;
d03     }

cf6     void update(int i, int x) {
1a7         int p = i + pot2;
3bc         tree[p] = x;
8b8         p >>= 1;
f77         while (p >= 1) {
280             tree[p] = max(tree[2*p], tree[2*p+1]);
8b8             p >>= 1;
6f4         }
4ad     }
092 }

a70 namespace HLD{
b24     vector<int> graph[MAX];
80f     int timer, h[MAX], ancestor[MAX], sz[MAX], pos[MAX],
base[MAX], peso[MAX];

696     void build_hld(int v, int p = -1, int f = 1) {
c06         base[pos[v] = timer++] = peso[v];
588         sz[v] = 1;
504         for (auto &i : graph[v]) if (i != p) {
983             ancestor[i] = v;
71b             h[i] = (i == graph[v][0] ? h[v] : i);
e76             build_hld(i, v, f); sz[v] += sz[i];

```

```

c51         if (sz[i] > sz[graph[v][0]] or graph[v][0] == p)
swap(i, graph[v][0]);
4e8     }
fd6         if (p*f == -1) build_hld(h[v] = v, -1, timer = 0);
bb3     }

1f8     void build(int root = 0) {
451         timer = 0;
295         build_hld(root);
160         SegTree::build(timer, base);
e95     }

f04     int query_path(int a, int b) {
11e         int res = 0;
9a1         while (h[a] != h[b]) {
aa1             if (pos[a] < pos[b]) swap(a, b);
30d             res = max(res, SegTree::query(pos[h[a]], pos[a]));
78b             a = ancestor[h[a]];
f86         }
9bc         if (pos[a] > pos[b]) swap(a, b);
bb7         res = max(res, SegTree::query(pos[a], pos[b]));
b50         return res;
852     }

081     void update_node(int i, int x) {
6e7         peso[i] = x;
2dc         SegTree::update(pos[i], x);
b94     }
95d }

0dd signed main(){
3b9     int n, q;
55a     cin >> n >> q;
04b     for (int i = 0; i < n; i++) cin >> HLD::peso[i];
6f5     for (int i = 1; i < n; i++){
ba2         int a, b;
0a8         cin >> a >> b;
e67         --a; --b;
770         HLD::graph[a].push_back(b);
ddb         HLD::graph[b].push_back(a);
364     }
b9a     HLD::build();
1f4     while(q--){
961         int type;
7b9         cin >> type;

```

```

821     if (type == 1){
8d2         int s, x;
f3c         cin >> s >> x;
351         s--;
847         HLD::update_node(s,x);
8e7     }
4e6     else {
ba2         int a, b;
0a8         cin >> a >> b;
450         a--; b--;
0ef         cout << HLD::query_path(a,b) << endl;
0ac     }
0a4 }
fba }

```

6.18 Query de Segtree Offline

```

a03 int n, tree[4*MAX];

d42 int query (int node, int l, int r, int a, int b){
d5e     if (b < l || r < a) return 0;
14b     if (l >= a && r <= b) return tree[node];
ee4     int m = (l+r)/2;
5a3     return query(node*2, l, m, a, b) + query(node*2+1, m+1, r, a,
b);
287 }

27f void update(int node, int l, int r, int i, int x){
791     if (i < l or i > r) return;
893     if (l == r) {
d6d         tree[node]=x;
505         return;
b74     }
ee4     int m = (l+r)/2;
3b2     update(node*2, l, m, i, x);
29a     update(node*2+1, m+1, r, i, x);
a16     tree[node] = tree[node*2] + tree[node*2+1];
d72 }

0dd signed main(){
5d6     winton;
45f     int q;
55a     cin >> n >> q;
f96     vector<pair<int,int>> a(n);
603     for (int i = 0; i < n; i++) {
8d6         cin >> a[i].first;

```

```

c92         a[i].second = i;
bd5     }
816     vector<tuple<int,int,int,int>> queries;
abf     for (int i = 0; i < q; i++){
12b         int l, r, m;
4c8         cin >> l >> m >> r;
0ef         if (a[l-1].first < m && r > 0){
205             queries.push_back ({m, l, l+r-1, i});
705         }
342     }
6a2     sort(rall(a));
7a3     sort(rall(queries));
bec     int pos = 0;
4cf     vector<int>ans(q,0);
5cc     for (auto [m,l,r,i] : queries){
730         while (pos < n && a[pos].first >= m){
// debug(m);
// debug(l);
// debug(r);
// debug(a[pos].second);
d85         update(1,0,n-1,a[pos].second,1);
657         pos++;
f08     }
a09     ans[i] = query(1,0,n-1,l,r);
// cout << tree[8] << " " << tree[9] << " ";
// for (int i = 5; i <= 7; i++)cout << tree[i] << " ";
// cout << endl;

088     }
16b     for (auto u : ans) cout << u << endl;
e2b }

```

6.19 Simspsons Paradox

```

//Trend or pattern that appears in different groups of data is
inconsistent
//(disappears or even reverses) with what we see when the groups are
combined.

//
//
//In this problem you are to construct an example of the dataset
illustrating the Simpson's paradox.0
//More specifically, let us assume (similarly to the example above)
that there are two teams who have been
//solving problems of N categories and the total number of problems
solved by each of the teams is M

```

```
//Let us denote the number of the problems in i-th category solved by
    Team X as ai, attempted by bi
//Similarly, let us define ci as the number of problems solved by Team
    Y in the i-th category and di as the number of problems attempted.
```

```
Odd signed main(){
14e     int n, m;
aa3     cin >> n >> m;
        // a1 = b1 = 1, isso da um ratio de 1 usando so uma questao
        // c1 = d1-1, isso da um ratio de 0.99 usando quantas questoes
        eu quiser
        // so resta calcular para as restantes das categorias

8dd     int d1 = m - 3*(n-1);
        //debug(d1);
4d4     int c1 = d1-1;
93a     cout << 1 << " " << 1 << " " << c1 << " " << d1 << endl;
b28     for (int i = 0; i < n-2; i++){
0cd         cout << 1 << " " << 2 << " " << 1 << " " << 3 << endl;
159     }
abc     int bn = m-((n-2)*2)-1;
5da     int dn = m-d1-((n-2)*3);
816     int an = (c1 - 1);
c56     int cn = 1;
393     cout << an << " " << bn << " " << cn << " " << dn << endl;
cbd }
```

6.20 Steiner Tree

```
// steiner: retorna o peso da menor arvore que cobre os vertices S
// get_steiner: retorna o valor minimo e as arestas de uma solucao
// se nao tiver solucao retorna LINF
//
// grafo nao pode ter pesos negativos
// se so tiver peso nas arestas/vertices pode deletar os vw/w no codigo
//
// k = |S|
//  $O(3^k * n + 2^k * m \log m)$ 

// otimizacao: joga um vertice x do S fora e pegue a resposta em
    dp[...][x] e reconstrua a arvore a partir dele
// ta comentado no codigo as mudancas necessarias

1a8 int n; // numero de vertices
c0d vector<pair<int, int>> g[MAX]; // {vizinho, peso}
```

```
920 ll d[1 << K][MAX]; // dp[mask][v] = arvore minima com o
    subconjunto mask de S e o vertice v
0b0 ll vw[MAX]; // peso do vertice

c8f ll steiner(const vector<int> &S) {
934     int k = S.size(); // k--;
dea     for (int mask = 0; mask < (1 << k); mask++) for(int v = 0; v <
n; v++) d[mask][v] = LINF;
6b8     for (int v = 0; v < n; v++) d[0][v] = vw[v];
254     for (int i = 0; i < k; ++i) d[1 << i][S[i]] = vw[S[i]];
042     for (int mask = 1; mask < (1 << k); mask++) {
b5b         for (int a = (mask - 1) & mask; a; a = (a - 1) & mask) {
638             int b = mask ^ a;
6bf             if (b > a) break;
84d             for (int v = 0; v < n; v++)
2e6                 d[mask][v] = min(d[mask][v], d[a][v] + d[b][v] -
vw[v]);
2ab         }
88c         priority_queue<pair<ll, int>> pq;
778         for (int v = 0; v < n; v++) {
6ad             if (d[mask][v] == LINF) continue;
5ca             pq.emplace(-d[mask][v], v);
f2e         }
265         while (pq.size()) {
a25             auto [ndist, u] = pq.top(); pq.pop();
dad             if (-ndist > d[mask][u]) continue;
c38             for (auto [idx, w] : g[u]) if (d[mask][idx] >
d[mask][u] + w + vw[idx]) {
679                 d[mask][idx] = d[mask][u] + w + vw[idx];
a2e                 pq.emplace(-d[mask][idx], idx);
07e             }
de5         }
a65     }
478     return d[(1 << k) - 1][S[0]]; // S[k]
        //para pegar o custo minimo para juntar todos os S's basta
        percorrer a mask-1 para todo no e ver qual o menor

704 }

d41 #warning se k=1 a solucao eh a folha isolada e a funcao retorna
    edg = {}
d41 #warning se k=0 crasha
4d1 pair<ll, vector<pair<int, int>>> get_steiner(const vector<int> &S) {
934     int k = S.size(); // k--;
8ec     ll ans = steiner(S);
c8d     vector<pair<int, int>> edg;
57f     stack<pair<int, int>> stk;
09b     stk.emplace((1 << k) - 1, S[0]); // S[k]
```

```

07d     while (!stk.empty()) {
c37         bool cont = 0;
9c6         auto [mask,u] = stk.top();stk.pop();
de2         if ((__builtin_popcount(mask) == 1 and u ==
S[(__bit_width(mask) - 1])) continue;
851         for (auto [idx, w] : g[u]){
bb4             if (d[mask][u] == d[mask][idx] + w + vw[u]) {
fc7                 edg.emplace_back(u, idx);
8ab                 stk.emplace(mask, idx);
a04                 cont = true;
c2b                 break;
342             }
ed9         }
3b5         if (cont) continue;
b5b         for (int a = (mask - 1) & mask; a; a = (a - 1) & mask) {
638             int b = mask ^ a;
1e8             if (d[mask][u] == d[a][u] + d[b][u] - vw[u]) {
be8                 stk.emplace(a, u);
c0a                 stk.emplace(b, u);
a04                 cont = true;
c2b                 break;
f52             }
d29         }
c5c         assert(!mask || cont);
2b1     }
be8     return {ans, edg};
cf6 }

```

6.21 Subtree Queries

```

5c1 const int MAXN = 2e5+7;

fff int tree[4*MAXN], tin[MAXN], tout[MAXN], flat[MAXN];
309 vector<vector<int>>>graph;
2dc int timer = 0;

955 void dfs(int v, int p){
237     tin[v] = ++timer;
284     for (auto u : graph[v]){
58d         if (u!=p){
412             dfs(u, v);
bb2         }
ab6     }
a66     tout[v] = timer;
03d }

```

```

7a2 int build(int node, int l, int r){
893     if (l == r){
019         return tree[node] = flat[l];
f94     }
ee4     int m = (l+r)/2;
174     return tree[node] = build(node*2, l, m) + build(node*2+1, m+1,
r);
97f }

d42 int query (int node, int l, int r, int a, int b){
d5e     if (b < l || r < a) return 0;
14b     if (l >= a && r <= b) return tree[node];
ee4     int m = (l+r)/2;
5a3     return query(node*2, l, m, a, b) + query(node*2+1, m+1, r, a,
b);
287 }

27f void update(int node, int l, int r, int i, int x){
791     if (i < l or i > r) return;
893     if (l == r) {
d6d         tree[node]=x;
505         return;
b74     }
ee4     int m = (l+r)/2;
3b2     update(node*2, l, m, i, x);
29a     update(node*2+1, m+1, r, i, x);
a16     tree[node] = tree[node*2] + tree[node*2+1];
d72 }

0dd signed main(){
3b9     int n, q;
55a     cin >> n >> q;
979     vector<int>arr(n+1);
78a     for (int i = 1; i <= n; i++){
5d3         cin >> arr[i];
370     }
8f4     graph.resize(n+1);
6f5     for (int i = 1; i < n; i++){
ba2         int a, b;
0a8         cin >> a >> b;
8fa         graph[a].push_back(b);
4c6         graph[b].push_back(a);
080     }
b23     dfs(1,1);
78a     for (int i = 1; i <= n; i++){
a0b         flat[tin[i]] = arr[i];
616     }

```

```

be6      build(1,1,n);

1f4      while(q--){
ffd          int op;
f99          cin >> op;
e55          if (op == 1){
d5f              int idx, valor;
066              cin >> idx >> valor;
931              update(1, 1, n, tin[idx], valor);
fcf          }
4e6          else {
8ab              int s;
a18              cin >> s;

16b              int ans = query (1,1,n,tin[s], tout[s]);
886              cout << ans << endl;
97e          }

d0f      }
a7b }

```

7 Strings

7.1 KMP

```

737      int n = (int)s.length();
d93      vector<int> pi(n);
6f5      for (int i = 1; i < n; i++) {
3f8          int j = pi[i-1];
a2c          while (j > 0 && s[i] != s[j])
25c              j = pi[j-1];
f36          if (s[i] == s[j])
04b              j++;
d2e          pi[i] = j;
a4e      }
81d      return pi;
cbb }

//implementacao minha
6cc vector<int> kmp(string s){
357     int n = s.size(), pfxlen = 0;
ca9     vector<int> a(n, 0);
6f5     for (int i = 1; i < n; i++){
e60         if (s[i] == s[pfxlen]) a[i] = ++pfxlen;

```

```

7f3         else if (pfxlen){
6b3             pfxlen = a[pfxlen-1];
169             i--;
4de         }
409         else a[i] = 0;
535     }
3f5     return a;
836 }

```

7.2 Manacher

//Manacher para achar todos os palindromes e depois reconstruir eles
//Nao e eficiente pois utiliza substr 0(size)

```

fd3 vector<int> manacher_odd(string s) {
163     int n = s.size();
0cb     s = "$" + s + "^";
1a9     vector<int> p(n + 2);
187     int l = 0, r = 1;
78a     for(int i = 1; i <= n; i++) {
a1d         p[i] = min(r - i, p[l + (r - i)]);
ce7         while(s[i - p[i]] == s[i + p[i]]) {
fbd             p[i]++;
dc2         }
03e         if(i + p[i] > r) {
26d             l = i - p[i];
a69             r = i + p[i];
4cc         }
283     }
fa1     return vector<int>(begin(p) + 1, end(p) - 1);
08d }

4e5 vector<int> manacher(string s) {
78f     string t;
7e7     for(auto c: s) {
870         t += string("#") + c;
478     }
9cc     auto res = manacher_odd(t + "#");
b50     return res;
cdc }

Odd signed main() {
5d6     winton;
ac0     string s;
a18     cin >> s;

```

```

bf9     vector<int> mnc = manacher(s);
35c     vector<pair<int,int>> palindromes;
63e     for (int i = 0; i < mnc.size(); i++){
1e8         if (mnc[i]-1 > 0){
8ad             palindromes.push_back({mnc[i]-1,i}); //guarda os
palindromes com {tamanho, centro}
ad1         }
b1c     }

668     sort(rall(palindromes)); //ordena pelos maiores (opcional ne)
603     for (auto &[size, centro] : palindromes){

04c         string pos;
3f0         int st = ((centro - size)/2);
ab5         pos = s.substr(st, size); //gera a substring daquele
palindrome
2dc     }
bf9 }

```

7.3 String Hashing

```

// Complexidades:
// construtor - O(|s|)
// operator() - O(1)

878 mt19937 rng((int)
chrono::steady_clock::now().time_since_epoch().count());

463 int uniform(int l, int r) {
a7f     uniform_int_distribution<int> uid(l, r);
f54     return uid(rng);
d9e }

9e0 template<int MOD> struct str_hash {
c63     static int P;
dcf     vector<ll> h, p;
ea8     str_hash(string s) : h(s.size()), p(s.size()) {
7a2         p[0] = 1, h[0] = s[0];
ad7         for (int i = 1; i < s.size(); i++)
84c             p[i] = p[i - 1]*P%MOD, h[i] = (h[i - 1]*P + s[i])%MOD;
1ef     }
af7     ll operator()(int l, int r) { // retorna hash s[l...r]
749         ll hash = h[r] - (l ? h[l - 1]*p[r - l + 1]%MOD : 0);
dfd         return hash < 0 ? hash + MOD : hash;
3ba     }
116 };

```

```

217 template<int MOD> int str_hash<MOD>::P = uniform(256, MOD - 1); //
1 > |sigma|

295 vector<int> rabin_karp(string const& s, string const& t) {
2b7     int S = s.size(), T = t.size();

d52     vector<int> p_pow(max(S, T));
b60     p_pow[0] = 1;
8db     for (int i = 1; i < (int)p_pow.size(); i++) p_pow[i] =
(p_pow[i-1] * P) % MOD;

404     vector<int> h(T + 1, 0);
a59     for (int i = 0; i < T; i++) h[i+1] = (h[i] + (t[i] - 'a' + 1)
* p_pow[i]) % MOD; //hash do texto (vetor a)

2d8     int h_s = 0;
535     for (int i = 0; i < S; i++) h_s = (h_s + (s[i] - 'a' + 1) *
p_pow[i]) % MOD; //hash da string s (vetor b)

a15     vector<int> occurrences;
fce     for (int i = 0; i + S - 1 < T; i++) {
d38         int cur_h = (h[i+S] + MOD - h[i]) % MOD;
7f5         if (cur_h == h_s * p_pow[i] % MOD)
occurrences.push_back(i);
77b     }
831     return occurrences;
78d }

//struct para um vetor de strings
//consegue retornar um vetor sem as strings que sao substrings de
outras
//consegue checar se o prefixo de uma string e o sufixo de outra

803 struct hashing {
2a9     int p_pow[MAX];
52b     vector<int> hashes[11];
3df     vector<string> strings;
1a8     int n;

f09     hashing(int _n, vector<string> _a) : n(_n), strings(_a) {}

0a8     void build(){
b60         p_pow[0] = 1;

2cf         for (int i = 1; i < MAX; i++) p_pow[i] = (p_pow[i-1] * P)
% MOD;

```

```

bde         for (int i = 0; i < n; i++) hashes[i] =
    calcHash(strings[i]);
e10     }

e10     vector<string> trim(){
86b         vector<string> ret;
            //funcao pra remover do vetor original as strings que sao
            substrings de outras
            //nao preciso delas pra fazer a verificacao depois
3c4         set<int> occurrences;
603         for (int i = 0; i < n; i++){
578             for (int j = 0; j < n; j++){
4ce                 if (i == j) continue;
680                 int S = strings[i].size(), T = strings[j].size();
d36                 int h_s = hashes[i][S];
568                 for (int k = 0; k + S - 1 < T; k++) {
df2                     int cur_h = (hashes[j][k+S] + MOD -
hashes[j][k]) % MOD;
f30                     if (cur_h == h_s * p_pow[k] % MOD)
occurrences.insert(i);
e4e                 }
b00             }
f98         }
603         for (int i = 0; i < n; i++){
851             if (occurrences.find(i) != occurrences.end()) continue;
07c             ret.push_back(strings[i]);
aa6         }
9ca         debug(occurrences);
edf         return ret;
391     }

5ae     vector<int> calcHash(string s){
fa3         vector<int> h(s.size() + 1, 0);
18f         for (int i = 0; i < s.size(); i++) h[i+1] = (h[i] + (s[i]
- 'a' + 1) * p_pow[i]) % MOD;
81c         return h;
a53     }

4c8     int get_hash(int l, int len, int idx){
488         int r = l + len;
915         return ((hashes[idx][r] + MOD - hashes[idx][l]) % MOD);
96f     }

166     bool equal_substrings(int len, int i, int j) {
2bf         if (!len) return true;

34c         int h1 = get_hash(0, len, i);

```

```

393         int h2 = get_hash(strings[j].size() - len, len, j);

21f         int left  = (h1 * p_pow[strings[j].size()-len]) % MOD;
eeb         int right = (h2) % MOD;
7ef         return left == right;
bda     }
890 };

//struct de hashing pra uma string so
803 struct hashing {
bfe     vector<int> h, p_pow;
ac0     string s;
1a8     int n;

e68     hashing(string _s) : n(_s.size()), s(_s), h(_s.size()+1, 0),
p_pow(_s.size(), 0) {}

0a8     void build(){
b60         p_pow[0] = 1;

25f         for (int i = 1; i < n; i++) p_pow[i] = (p_pow[i-1] * P) %
MOD;
18f         for (int i = 0; i < s.size(); i++) h[i+1] = (h[i] + (s[i]
- 'a' + 1) * p_pow[i]) % MOD;
6c1     }

77c     int get_hash(int l, int len){
488         int r = l + len;
9c5         return ((h[r] + MOD - h[l]) % MOD);
2b2     }

a15     bool equal_substrings(int len) {
2bf         if (!len) return true;

925         int h1 = get_hash(0, len);
8f8         int h2 = get_hash(n - len, len);

ba9         int left  = (h1 * p_pow[n-len]) % MOD;
eeb         int right = (h2) % MOD;
7ef         return left == right;
c7c     }
e94 };

//double mod hashing
803 struct hashing {
ac0     string s;

```

```

1a8      int n;
7b9      vector<int> h1, h2;
11e      vector<int> p1, p2;

502      hashing(const string &_s) : s(_s) {
40c          n = (int)s.size();
d13          h1.assign(n+1, 0);
667          h2.assign(n+1, 0);
918          p1.assign(n+1, 0);
ef0          p2.assign(n+1, 0);
138      }

0a8      void build() {
c4b          p1[0] = 1;
bf6          p2[0] = 1;
78a          for (int i = 1; i <= n; i++) {
382              p1[i] = (p1[i-1] * P1) % MOD1;
7c1              p2[i] = (p2[i-1] * P2) % MOD2;
3c9          }
603          for (int i = 0; i < n; i++) {
f06              int val = (s[i] - 'a' + 1);
420              h1[i+1] = (h1[i] + val * p1[i]) % MOD1;
6fe              h2[i+1] = (h2[i] + val * p2[i]) % MOD2;
9e2          }
231      }

67d      pair<int,int> get_hash(int l, int len) {
488          int r = l + len;
a93          int x1 = ((h1[r] - h1[l]) + MOD1) % MOD1;
436          int x2 = ((h2[r] - h2[l]) + MOD2) % MOD2;
199          return {x1, x2};
9cc      }
d3b };

```

7.4 StringStream

```

141 vector<string> split_sentence(string s){
942     stringstream ss(s);

6de     string word;

622     vector<string> v;

dac     while(getline(ss, word, ' ')){
e6c         v.push_back(word);
8e0     }

```

```

6dc         return v;
c7e     }

```

8 Extra

8.1 build.sh

```
#!/bin/bash

# 1. Check if a file was provided
if [ -z "$1" ]; then
    echo "Usage: $0 <source_file.cpp>"
    exit 1
fi

SOURCE_FILE="$1"

# 2. Extract the filename without the extension and append .out
# Adding a distinct extension separates it from the source file in
# terminal autocomplete
OUTPUT_BINARY="${SOURCE_FILE%.*}.out"

# 3. Check if the executable exists and is up-to-date
# -nt checks if a file is strictly newer. Negating it skips compilation
# if the binary is newer than or the exact same age as the source.
if [ -f "$OUTPUT_BINARY" ] && [ ! "$SOURCE_FILE" -nt "$OUTPUT_BINARY" ]; then
    echo "Already updated. Run it with: ./$OUTPUT_BINARY"
    exit 0
fi

# 4. Define your compiler and flags
CXX="g++"
CXXFLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer -g
-Wall -Wshadow -std=c++23 -Wno-unused-result -Wno-sign-compare
-Wno-char-subscripts"

echo "Compiling $SOURCE_FILE..."

# 5. Execute the compile command
$CXX $CXXFLAGS "$SOURCE_FILE" -o "$OUTPUT_BINARY"

# 6. Check if compilation was successful
if [ $? -eq 0 ]; then
    echo "Success! Run it with: ./$OUTPUT_BINARY"
else
    echo "Compilation failed."
fi
```

8.2 compressao.cpp

```
//compressao de coordenada para vector

vector<int> v(n);
vector<int> todos;
for (int i = 0; i < n; i++){
    cin >> v[i];
    todos.push_back(v[i]);
}

sort(all(todos));
todos.erase(unique(all(todos)), todos.end());
int m = todos.size();

vector<int> comp(n);
for (int i = 0; i < n; i++){
    comp[i] = lower_bound(all(todos), v[i]) - todos.begin();
}

template <typename T>
struct Compressor {
    vector<T> coords;

    // 1. Add all coordinates from queries and updates
    void add(T x) {
        coords.push_back(x);
    }

    // 2. Sort and remove duplicates
    void build() {
        sort(coords.begin(), coords.end());
        coords.erase(unique(coords.begin(), coords.end()),
            coords.end());
    }

    // 3. Get compressed index (0-indexed)
    int get(T x) {
        return lower_bound(coords.begin(), coords.end(), x) -
            coords.begin();
    }

    // 4. Get original value from compressed index
    T orig(int idx) {
        return coords[idx];
    }
}
```

```
// 5. Get total number of unique coordinates
int size() {
    return coords.size();
}
};
```

8.3 debug.cpp

```
//Debug
```

```
/*mais simples de todos, imprime desformatado na linha*/
void dbg_out() { cerr << endl; }
template<typename H, typename... T>
void dbg_out(H h, T... t) { cerr << ' ' << h; dbg_out(t...); }

#define DEBUG

#if defined(DEBUG)
#define winton (void)0
#define debug(...) cerr << #__VA_ARGS__ << ' ';
    dbg_out(__VA_ARGS__);
#else
#define winton ios_base::sync_with_stdio(false), cin.tie(NULL)
#define debug(...) (void)0
#endif
//=====//

/*debug com cor (longo mas mt forte, imprime qualquer container
tirando coisa com .pop() -> adicionar depois)*/
namespace dbg {
    const char* const RESET      = "\033[0m";
    const char* const BOLD_BLUE  = "\033[1;34m";
    const char* const YELLOW     = "\033[33m";
    const char* const BOLD_WHITE = "\033[1;37m";

    template<typename T1, typename T2>
    ostream& operator<<(ostream& os, const pair<T1, T2>& p) { return
        os << '{' << p.first << ", " << p.second << '}' ; }

    template<typename T_container, typename T = typename
        enable_if<!is_same_v<T_container, string> &&
        !is_same_v<T_container, string_view>, typename
        T_container::value_type>::type>
    ostream& operator<<(ostream& os, const T_container& v) {
        os << '{';
```

```
        bool first = true;
        for (const T& x : v) { os << (first ? "" : ", ") << x, first =
            false; }
        return os << '}' ;
    }

    void debug_out(string_view) { cerr << endl; }
    template<typename H, typename... T>
    void debug_out(string_view s, H h, T... t) {
        auto cpos = s.find(',');
        cerr << YELLOW << s.substr(0, cpos) << RESET << " = ";
        cerr << BOLD_WHITE << h << RESET;
        if constexpr (sizeof...(t) > 0) {
            cerr << ", ";
            auto nx = s.find_first_not_of(" \t\n\r", cpos + 1);
            debug_out(s.substr(nx), t...);
        } else {
            cerr << endl;
        }
    }
}
using namespace dbg;

#define DEBUG

#if defined(DEBUG)
#define winton (void)0
#define debug(...) cerr << BOLD_BLUE << "[" << __func__ << ":" <<
    __LINE__ << "]" << RESET << " "; debug_out(__VA_ARGS__,
    __VA_ARGS__)
#else
#define winton
    ios_base::sync_with_stdio(false), cin.tie(NULL), cout.tie(NULL)
#define debug(...) (void)0
#endif
//=====//

/*debug mais simples, so imprime vars mas imprime organizado na linha,
menor e mais facil de codar*/
void debug_out(string s, int line) {cerr << endl; }
template<typename H, typename... T>
void debug_out(string s, int line, H h, T... t){
    do{
        cerr << line << ": " << s[0]; s = s.substr(1);
    }
    while (sz(s) and s[0] != ',');
```

```

    cerr << " = " << h;
    debug_out(s, line, t...);
}

#define DEBUG

#if defined(DEBUG)
#define winton (void)0
#define debug(...) debug_out(#__VA_ARGS__, __LINE__, __VA_ARGS__)
#else
#define winton ios_base::sync_with_stdio(false), cin.tie(NULL)
#define debug(...) (void)0
#endif
//=====//

/*debug foda mas sem cor*/
namespace dbg {
    template<typename T1, typename T2>
    ostream& operator<<(ostream& os, const pair<T1, T2>& p) { return
        os << '{' << p.first << ", " << p.second << '}''; }

    template<typename T_container, typename T = typename
        enable_if<!is_same_v<T_container, string> &&
        !is_same_v<T_container, string_view>, typename
        T_container::value_type>::type>
    ostream& operator<<(ostream& os, const T_container& v) {
        os << '{';
        bool first = true;
        for (const T& x : v) { os << (first ? "" : ", ") << x, first =
            false; }
        return os << '}'';
    }

    void debug_out(string_view) { cerr << endl; }
    template<typename H, typename... T>
    void debug_out(string_view s, H h, T... t) {
        auto cpos = s.find(',');
        cerr << s.substr(0, cpos) << " = " << h;
        if constexpr (sizeof...(t) > 0) {
            cerr << ", ";
            auto nx = s.find_first_not_of(" \t\n\r", cpos + 1);
            debug_out(s.substr(nx), t...);
        } else {
            cerr << endl;
        }
    }
}

```

```

}
using namespace dbg;

#define DEBUG

#if defined(DEBUG)
#define winton (void)0
#define debug(...) cerr << "[" << __func__ << ":" << __LINE__ <<
    "]" << " "; debug_out(#__VA_ARGS__, __VA_ARGS__)
#else
#define winton
    ios_base::sync_with_stdio(false), cin.tie(NULL), cout.tie(NULL)
#define debug(...) (void)0
#endif
//=====//

/*generic printer de todas as estruturas*/
// Generic printer for vector<T>
template <typename T>
ostream &operator<<(ostream &os, const vector<T> &v) {
    for (size_t i = 0; i < v.size(); ++i) {
        os << v[i];
        if (i + 1 < v.size()) os << ' ';
    }
    return os;
}

// Specialized printer for vector<vector<T>> (matrix)
template <typename T>
ostream &operator<<(ostream &os, const vector<vector<T>> &matrix) {
    for (size_t i = 0; i < matrix.size(); ++i) {
        os << matrix[i];
        if (i + 1 < matrix.size()) os << '\n';
    }
    return os;
}

//-----PAIR-----//
template<class A, class B> ostream& operator<<(ostream& os, const
    pair<A, B>& p) {
    os << "(" << p.first << ", " << p.second << ")";
    return os;
}
//-----VECTOR-----//
template<typename T> ostream& operator<<(ostream& os, const vector<T>&

```

```

    vec) {
        os << "[ ";
        for(const auto& elem : vec) {
            os << elem << " ";
        }
        os << "]";
        return os;
    }
//-----SET-----//
template<typename T> ostream& operator<<(ostream& os, const set<T>& s)
{
    os << "{ ";
    for(const auto& elem : s) {
        os << elem << " ";
    }
    os << "}";
    return os;
}
//-----MULTISET-----//
template<typename T> ostream& operator<<(ostream& os, const
multiset<T>& s) {
    os << "{ ";
    for(const auto& elem : s) {
        os << elem << " ";
    }
    os << "}";
    return os;
}
//-----QUEUE-----//
template<typename T> ostream& operator<<(ostream& os, queue<T> q) {
    // Print each element in the queue
    os << "{ ";
    while (!q.empty()) {
        os << q.front() << " ";
        q.pop();
    }
    os << "}";
    // Print a newline at the end
    return os;
}
//-----DEQUEUE-----//
template<typename T> ostream& operator<<(ostream& os, deque<T> q) {
    // Print each element in the queue
    os << "{ ";
    while (!q.empty()) {
        os << q.front() << " ";
        q.pop_front();
    }
}

```

```

    }
    os << "}";
    // Print a newline at the end
    return os;
}
//-----STACK-----//
template<typename T> ostream& operator<<(ostream& os, stack<T> q) {
    // Print each element in the queue
    os << "{ ";
    while (!q.empty()) {
        os << q.top() << " ";
        q.pop();
    }
    os << "}";
    // Print a newline at the end
    return os;
}
//-----PRIORITY_QUEUE-----//
template<typename T> ostream& operator<<(ostream& os,
priority_queue<T> q) {
    // Print each element in the queue
    os << "{ ";
    while (!q.empty()) {
        os << q.top() << " ";
        q.pop();
    }
    os << "}";
    // Print a newline at the end
    return os;
}
//-----MAP-----//
template<typename K, typename V> ostream& operator<<(ostream& os,
const map<K, V>& m) {
    os << "{ ";
    for(const auto& pair : m) {
        os << pair.first << " : " << pair.second << ", ";
    }
    os << "}";
    return os;
}
}

template<typename T>
using min_pq = priority_queue<T, vector<T>, greater<T>>;
template<typename T> ostream& operator<<(ostream& os, min_pq<T> q) {
    // Print each element in the queue
    os << "{ ";
    while (!q.empty()) {

```

```

        os << q.top() << " ";
        q.pop();
    }
    os << "}";
    // Print a newline at the end
    return os;
}

```

8.4 hash.sh

```

#!/bin/bash
s=() n=0
while IFS= read -r l || [[ -n "$l" ]]; do
    ((n++))

    o=$(grep -o "{" <<< "$l" | wc -l)
    c=$(grep -o "}" <<< "$l" | wc -l)

    if (( c > 0 && ${#s[@]} > 0 )); then
        start=${s[-1]}
    else
        start=$n
    fi

    for ((i=0; i<c; i++)); do
        if (( ${#s[@]} > 0 )); then unset 's[-1]'; fi
    done

    for ((i=0; i<o; i++)); do
        s+=($n)
    done

    h=$(sed -n "${start},${n}p" "$l" | sed '/^#/d' | cpp -dD -P
        -fpreprocessed 2>/dev/null | tr -d '[:space:]' | md5sum | cut
        -c-3)

    printf "%s %s\n" "$h" "$l"
done < "$1"

```

8.5 lambda.cpp

```

long long ans = 0;

auto lambda_padrao = [&](int x, int y) -> int {

```

```

    return x + y;
};

auto lambda_generica = [&](const auto &v) {
    return (int)v.size();
};

auto lambda_recurativa = [&](auto &&self, int x) -> int {
    if (x <= 1) return 1;
    return x * self(self, x - 1);
};

function<void(int, int)> dfs = [&](int v, int p) {
    for (auto u : graph[v]){
        if (u == p) continue;
        dfs(u,v);
    }
};

auto reduce = [&](const multiset<string> &ms) -> multiset<int> {
    multiset<int> mi;
    for (const auto &s : ms) {
        if (s.size() > 1) {
            mi.insert((int)s.size());
            ans++;
        } else {
            mi.insert(s[0] - '0');
        }
    }
    return mi;
};

```

8.6 new.sh

```

#!/bin/bash

# 1. Define where your template file lives
# Change "template.cpp" to whatever your actual template file is named
TEMPLATE_FILE="$HOME/ACE/resources/extra/template.cpp"

# 2. Check if the user provided a name for the new file
if [ -z "$1" ]; then
    echo "Usage: $0 <new_file_name>"
    exit 1
fi

```

```
TARGET_FILE="$1"

# 3. Check if the template actually exists
if [ ! -f "$TEMPLATE_FILE" ]; then
    echo "Error: Template not found at $TEMPLATE_FILE"
    exit 1
fi

# 4. Safety check: Prevent overwriting an existing file
if [ -f "$TARGET_FILE" ]; then
    echo "Error: '$TARGET_FILE' already exists! Aborting."
    exit 1
fi

# 5. Perform the copy operation
cp "$TEMPLATE_FILE" "$TARGET_FILE"

echo "Success! Created '$TARGET_FILE' from template."
```

8.7 prefix sum 2d.cpp

```
//Prefix Sum 2D

prefix[i][j]= prefix[i-1][j]+prefix[i][j-1]-prefix[i-1][j-1]+arr[i][j]
```

8.8 random.cpp

```
#include <random>
#include <chrono>

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution<long long> dist(0, n);//valores min e max

int x = dist(rng);
```

8.9 template.cpp

```
#include <bits/stdc++.h>
using namespace std;

template <typename T, typename U>
istream& operator>>(istream& is, pair<T, U>& p) {
```

```
    return is >> p.first >> p.second;
}

template <typename T>
istream& operator>>(istream& is, vector<T>& v) {
    for(auto& elem: v){
        is >> elem;
    }
    return is;
}

template <typename T, typename U>
ostream& operator<<(ostream& os, const std::pair<T, U>& p) {
    return os << "{" << p.first << ", " << p.second << "}";
}

template <typename T>
concept IterableContainer = requires(T t) {
    begin(t);
    end(t);
} && !is_convertible_v<T, string_view>;

template<IterableContainer Container>
ostream& operator<<(ostream& os, const Container& c) {
    os << "[";
    bool first = true;
    for(const auto& elem : c){
        if(!first) os << ", ";
        os << elem;
        first = false;
    }
    return os << "]";
}

#define fastio ios_base::sync_with_stdio(false), cin.tie(nullptr);
#define int long long
#define ld long double
#define endl '\n'

signed main(){
    fastio;

    return 0;
}
```